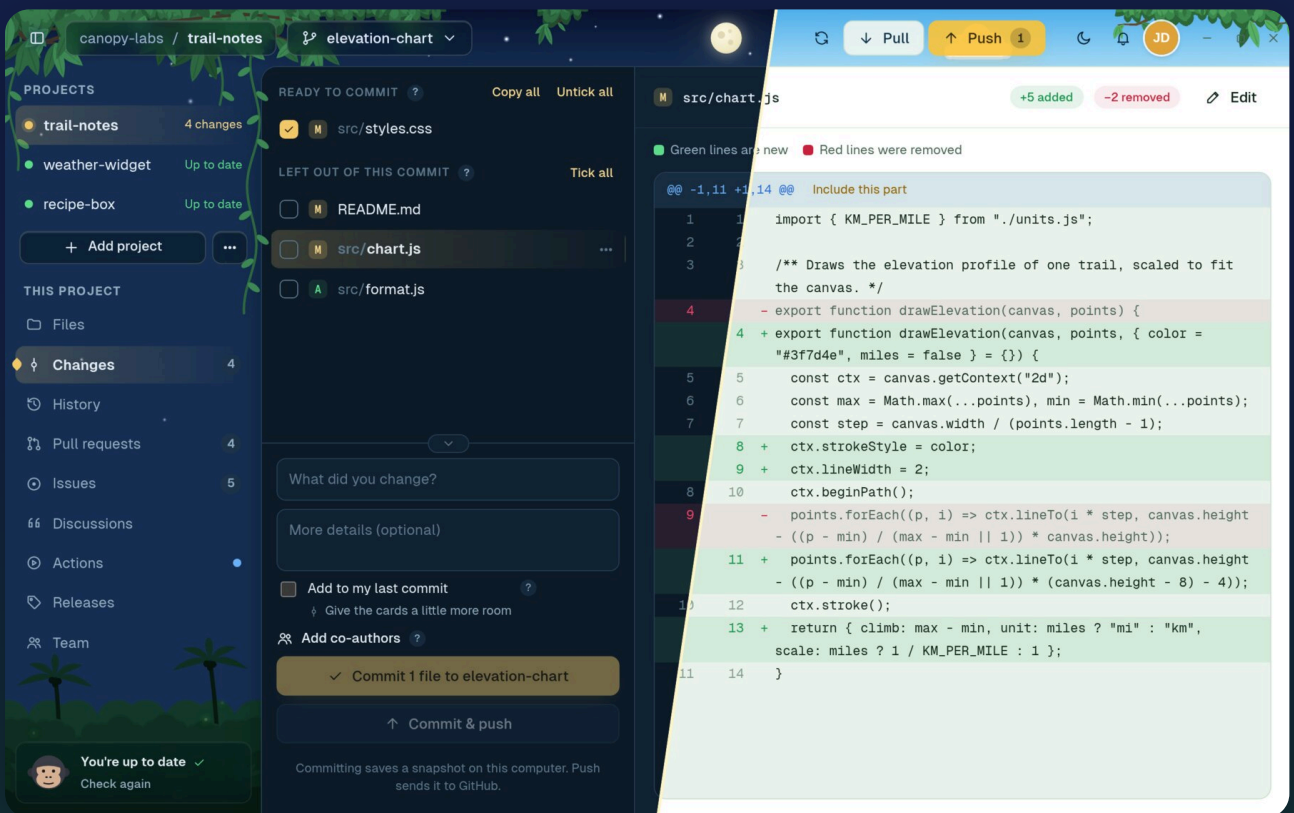




The GIT Monkey Guide

Everything you can do in GIT Monkey, page by page: committing, branches, pull requests, issues, discussions, Actions, releases and working as a team.



Contents

- 01 Welcome to GIT Monkey
- 02 Install
- 03 Getting started
- 04 Projects and the sidebar
- 05 Changes: committing your work
- 06 Files: browse, search and edit
- 07 Push, pull and staying in sync
- 08 Branches and put-aside changes
- 09 History
- 10 Pull requests
- 11 Issues
- 12 Discussions
- 13 Actions
- 14 Releases
- 15 Team
- 16 When something goes wrong
- 17 Safety nets
- 18 Settings and accounts
- 19 Keyboard shortcuts
- 20 FAQ
- 21 Getting help

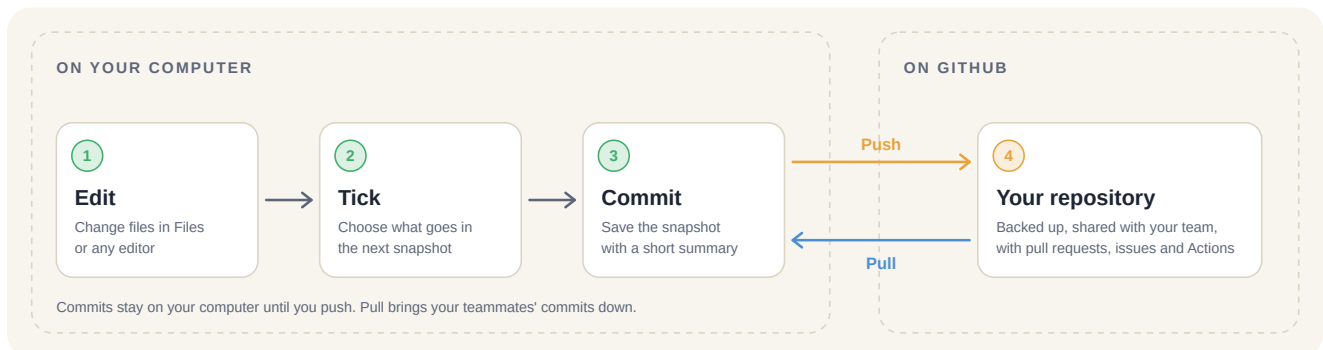
Welcome to GIT Monkey

Git is powerful, but it expects you to already know Git. GIT Monkey is built for everyone else: students, designers, writers and developers who would rather see their work than memorise commands.

- **See what changed.** Every edit is shown in color, line by line, before you save it.
- **Understand every step.** Buttons say what they do, and a `?` next to anything unfamiliar explains it.
- **Recover from mistakes.** Most everyday actions can be undone from the message that follows them, and deleted files wait in a Trash for 30 days.
- **Get unstuck fast.** When Git fails, GIT Monkey says what went wrong in plain English and, for common problems, offers a one-click fix.

Under the hood it runs the real `git` and `gh` (GitHub CLI) on your computer, so everything it does matches the command line exactly. Your projects stay ordinary Git repositories that work with any other tool, and there is no GIT Monkey account or server between you and GitHub.

How it works



That is the whole loop. **Edit** your files, **tick** the changes that belong together, and **commit** them as a snapshot with a short summary. Commits live on your computer until you **Push** them to GitHub; **Pull** brings down what your teammates pushed.

Git terms in plain English

GIT Monkey avoids jargon where it can, but you will see these words on GitHub and in tutorials:

Term	Meaning	Where to find it
Repository (repo)	A project folder whose history Git keeps	Each entry under Projects in the sidebar

Term	Meaning	Where to find it
Commit	A saved snapshot of your files, with a summary	Changes → Commit
Staging	Choosing which changes go into the next commit	Ticking files in Changes
Push / Pull	Send your commits to GitHub / get other people's	Buttons in the top bar
Fetch	Check GitHub for news without touching your files	The refresh button; also automatic
Branch	A separate line of work, so you can experiment safely	Branch menu in the top bar
Merge	Combine one branch's commits into another	Combine in the branch menu, or Merge on a pull request
Merge conflict	Two people changed the same lines and Git needs a decision	Red files in Changes
Stash	Unfinished changes put aside for later	Put my changes aside / Put-aside changes
Clone	Download a copy of a GitHub repository	Add project → Download from GitHub
Pull request (PR)	A request to merge a branch, so others can review it	Pull requests
Issue	A bug report, idea or task on GitHub	Issues
Discussion	A forum thread for questions and ideas	Discussions
Actions	Automatic jobs (tests, builds) GitHub runs after a push	Actions
Release / Tag	A named, finished version such as <code>v1.0.0</code>	Releases

Install

1. Download GIT Monkey

Pick the file for your system from the [latest release](#). The **Install GIT Monkey** setup is the easiest option: it installs, updates and removes the app for you.

System	Recommended	Alternatives
Windows 10/11	<code>Install-GIT-Monkey_<version>_x64.exe</code>	<code>..._x64-setup.exe</code> , <code>..._x64_en-US.msi</code> , or <code>..._x64-portable.zip</code> to run without installing
macOS (Apple Silicon and Intel)	<code>Install-GIT-Monkey_<version>_universal.dmg</code>	<code>GIT.Monkey_<version>_universal.dmg</code>
Linux (any distribution)	<code>Install-GIT-Monkey_<version>_amd64.AppImage</code>	<code>..._amd64.AppImage</code> to run directly
Debian, Ubuntu, Mint		<code>sudo apt install ./GIT.Monkey_*_amd64.deb</code>
Fedora, openSUSE		<code>sudo dnf install ./GIT.Monkey-*_x86_64.rpm</code>

To run an AppImage directly, make it executable first:

```
chmod +x GIT.Monkey_*_amd64.AppImage
./GIT.Monkey_*_amd64.AppImage
```

Note

GIT Monkey is not code-signed yet, so the first launch may show a warning. The app is safe to use, and every download lists a SHA-256 checksum on GitHub. - **Windows** (SmartScreen): click **More info**, then **Run anyway**. - **macOS** (Gatekeeper): open **System Settings → Privacy & Security** and click **Open Anyway**. If macOS says the app is damaged, run `xattr -dr com.apple.quarantine "/Applications/GIT Monkey.app"` and open it again.

2. Install Git and the GitHub CLI

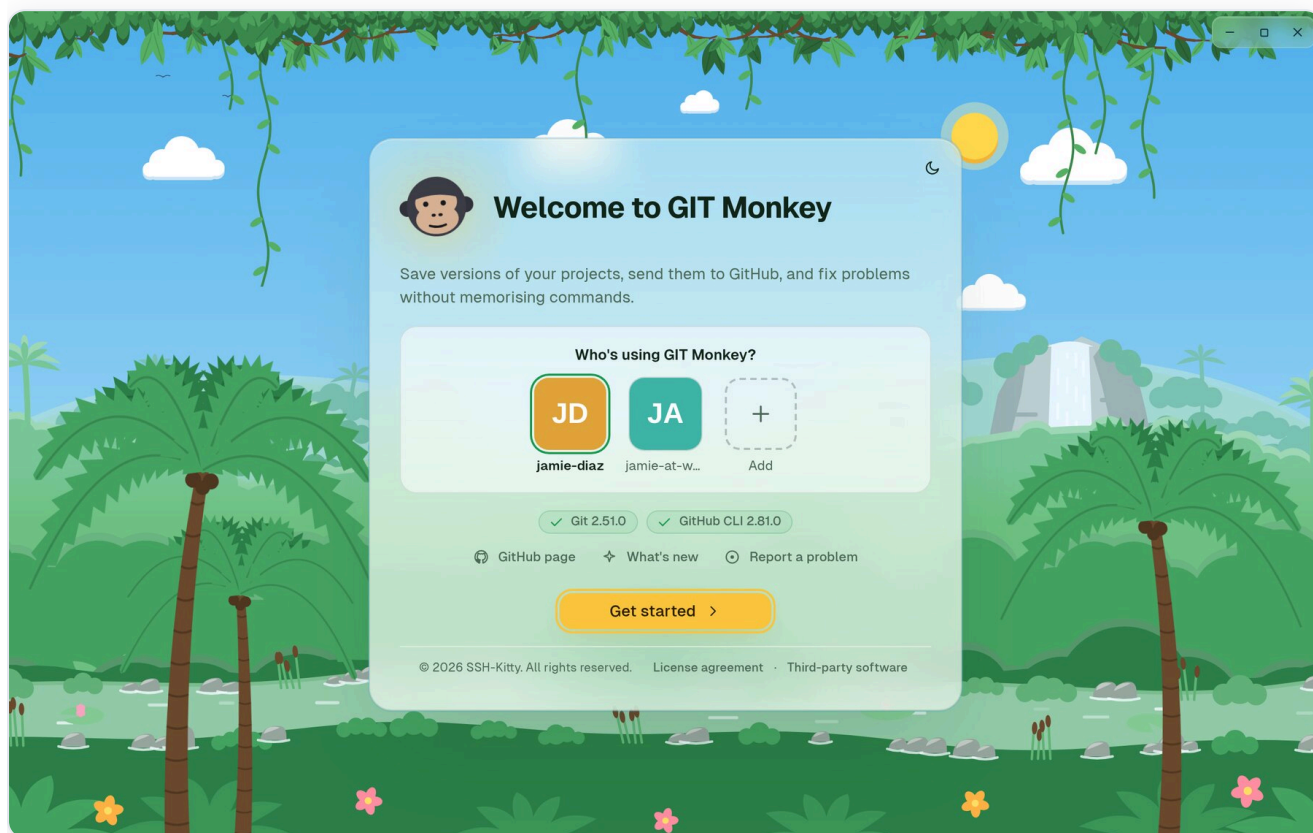
GIT Monkey relies on two free tools to do the actual work. It checks for both at startup and tells you if either is missing.

Tool	Windows	macOS	Linux
Git	<pre>winget install --id Git.Git -e</pre>	<pre>xcode-select --install</pre>	<pre>sudo apt install git</pre> (or your package manager)
GitHub CLI (gh)	<pre>winget install --id GitHub.cli -e</pre>	<pre>brew install gh</pre>	Installation guide

3. Stay up to date

GIT Monkey updates itself. The card at the bottom of the sidebar shows your version and checks for a new one at startup and every six hours. When an update is available, click **Update to vX.Y.Z**, review what's new, and choose **Update and restart**. Your projects and settings are kept.

Getting started



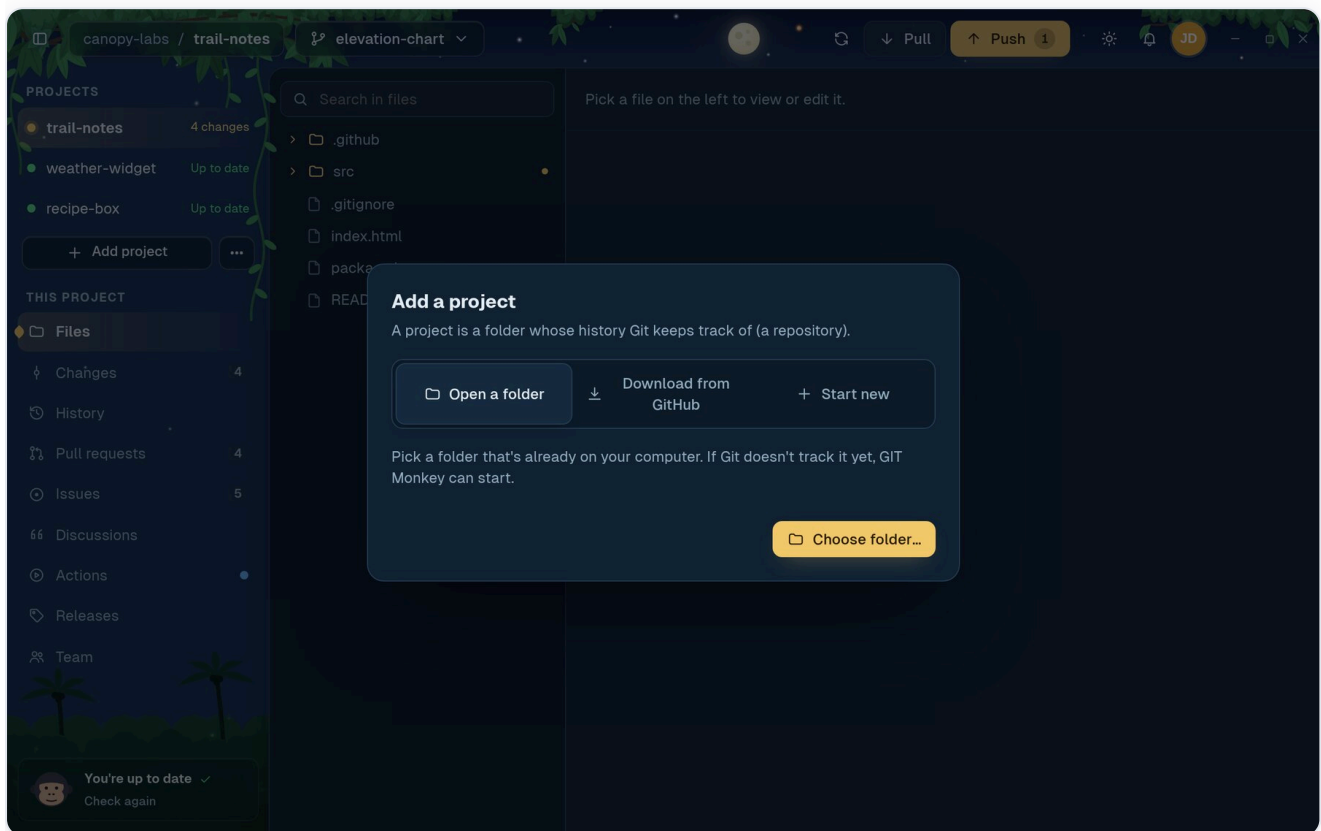
The welcome screen in the day theme: account picker, tool checks and Get started

1. **Open GIT Monkey.** The welcome screen confirms that Git and the GitHub CLI are installed. If one is missing, it shows the install command; run it and click **Check again**.
2. **Sign in with GitHub.** Click **Sign in with GitHub**, then **Open GitHub to enter the code**, paste the code shown and approve. The welcome screen notices when you're done. You only do this once: the sign-in is shared with the GitHub CLI, and Git is set up to use it for push and pull.
3. **Click Get started.** You can skip the welcome screen on later launches in **Settings → Behavior**.
4. **Add a project** with the **Add project** button in the sidebar (see [Projects](#)).
5. **Make your first commit.** Change a file, open **Changes**, tick it, write a summary such as "Fix typo in README", and click **Commit**. Then click **Push** to send it to GitHub.

Tip

If Git doesn't know your name yet, your first commit shows **"Git doesn't know your name yet"** with a **Use my GitHub name** button. One click fixes it, and you can change it later in [Settings → Commit identity](#).

Projects and the sidebar



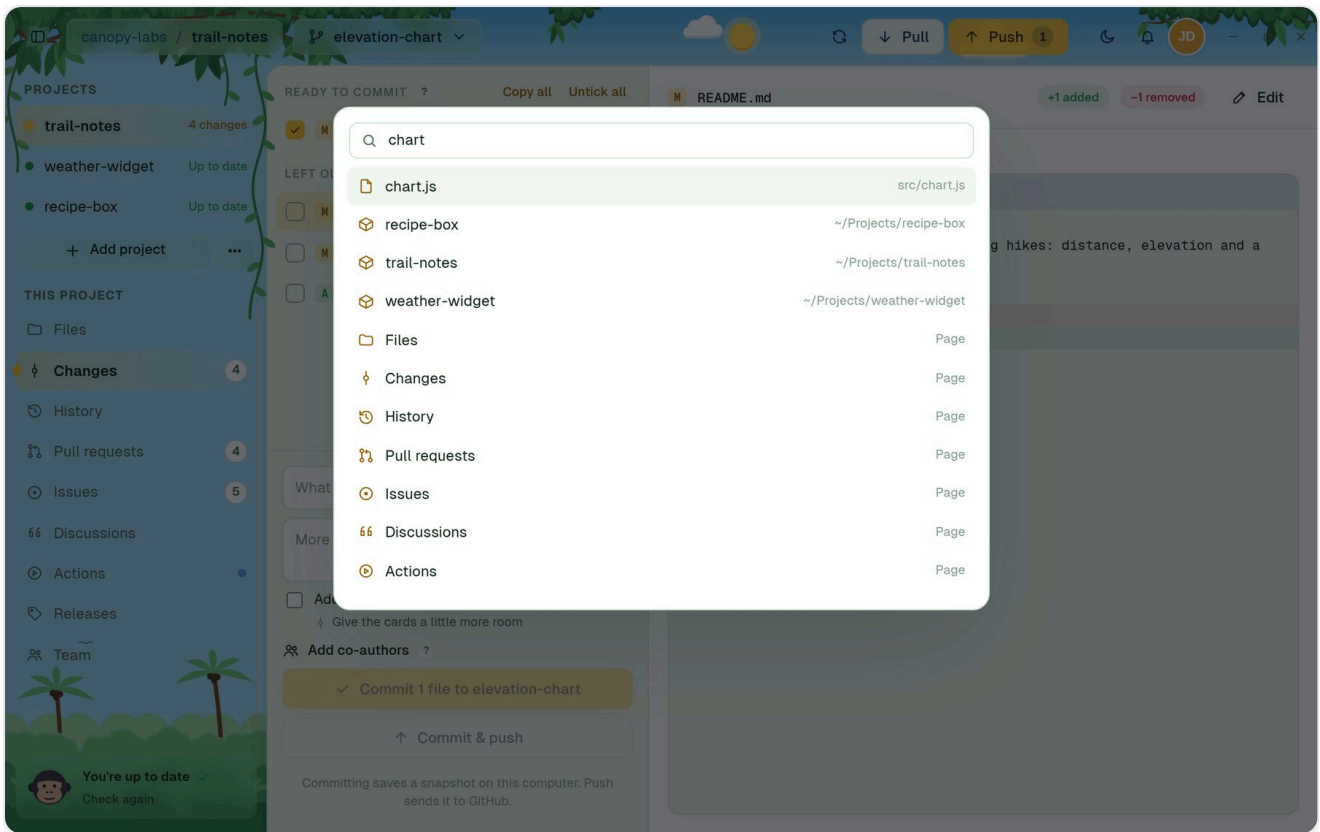
The Add a project dialog in the night theme: open a folder, download from GitHub, or start new

Add project offers three ways in:

- **Open a folder** already on your computer. If Git isn't tracking it yet, GIT Monkey offers to **start tracking** it. Your files are left exactly as they are.
- **Download from GitHub** (clone). Search your repositories, or type `owner/name` to download any public one.
- **Start new**. Give it a name and description; GIT Monkey creates the folder with a README and a first commit, and can publish it to GitHub as a **private** or **public** repository.

Each project in the sidebar has a status dot: **green** up to date, **yellow** changes or commits waiting, **blue** new commits to pull, **red** a merge conflict or a missing folder. Drag projects to reorder them. Right-click a project (or use ...) to **Lock** it, **Open in terminal**, **Open in VS Code**, **Show in folder**, **Copy path**, **Remove from list** or **Move to trash**.

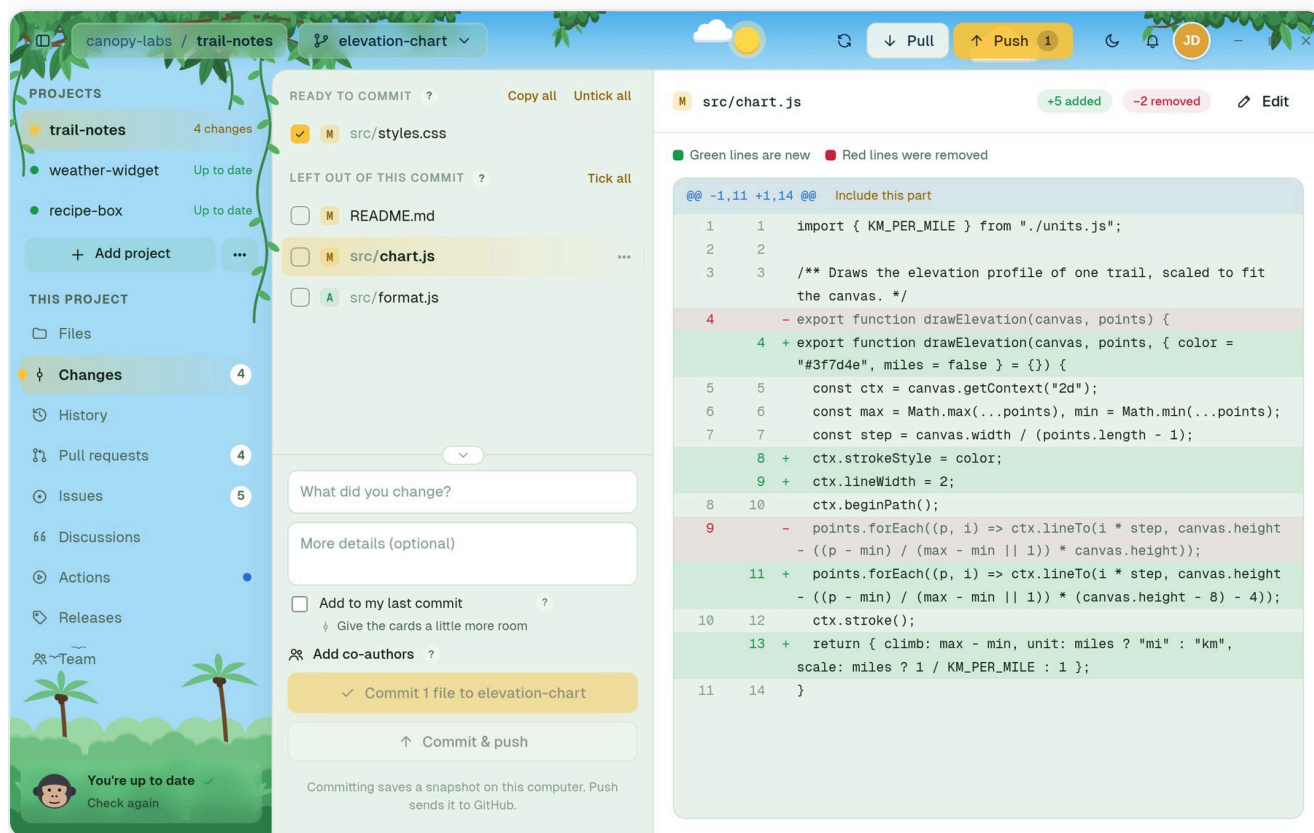
Below that are the project's pages: Files, Changes, History, Pull requests, Issues, Discussions, Actions, Releases and Team. Press **1** to **9** to jump between them, and drag them into the order you prefer. Counters show changed files, open pull requests and issues, and teammates online; the dot on Actions turns blue while a run is going and red when the last one failed.



Quick Open in the day theme, finding a file by typing part of its name

Press **K** for **Quick Open**: type part of a name to jump to any page, project or file. When someone invites you to a repository, an **Invites for you** card appears in the sidebar; **Accept** it and GIT Monkey offers to download the project straight away.

Changes: committing your work



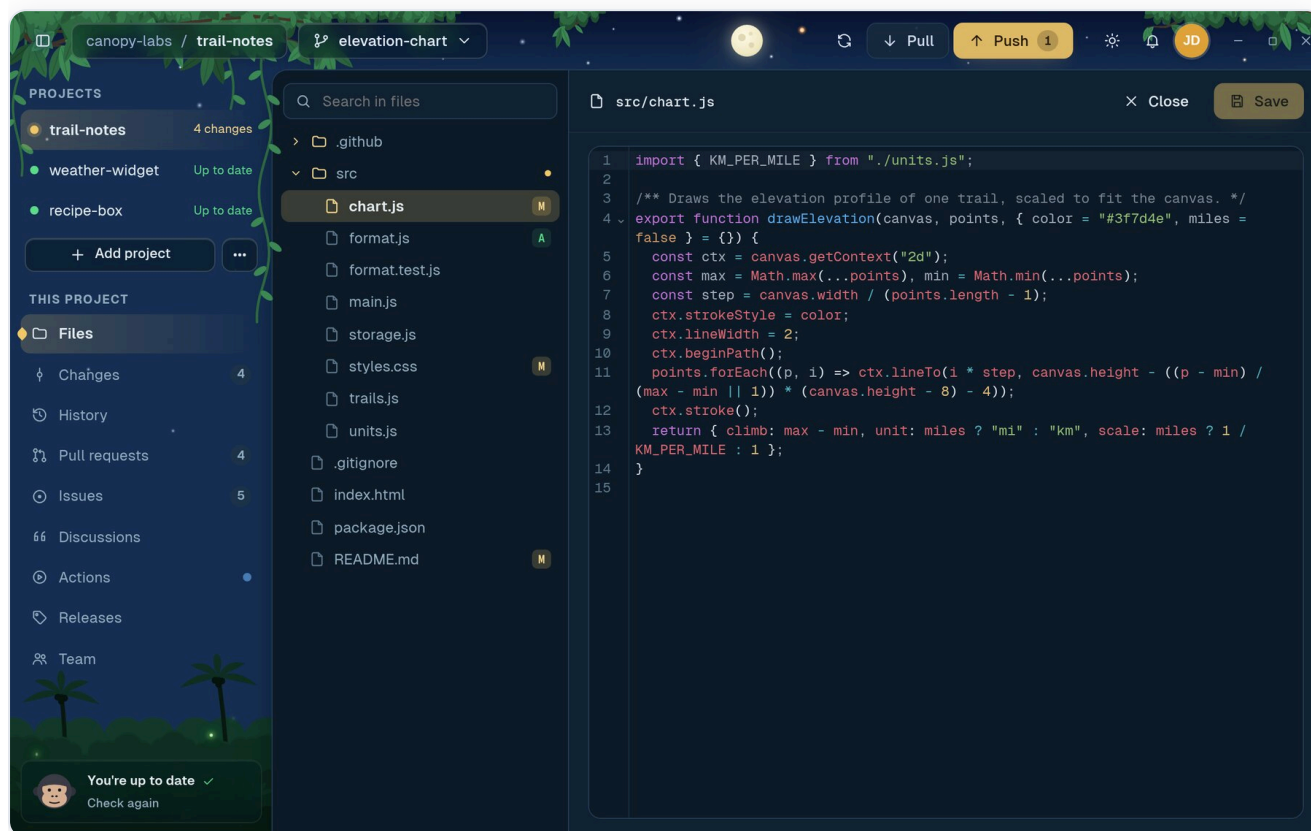
The Changes page in the day theme, showing a colored diff

Every file you've changed appears here, whether you edited it in GIT Monkey or another editor. A letter shows what happened to it: **M** modified, **A** added, **D** deleted, **R** renamed, **U** conflict.

- **Ready to commit** lists the ticked files; **Left out of this commit** lists the rest. Tick a file, press **Space**, or use **Tick all** / **Untick all**. Ctrl+click, Shift+click or drag to pick several at once.
- Click a file to see its changes: **green lines are new, red lines were removed**.
- **Include just part of a file**. Every block of changes has an **Include this part** link, so one file can be split across two commits.
- Write a **summary** and an optional description, then click **Commit (Ctrl+Enter)**, or **Commit & push** to do both at once. Your draft message is kept if you switch pages.
- **Add co-authors** to credit the people you worked with. **Add to my last commit** folds the ticked files into your newest commit while it is still unpushed.
- The ... menu on a file offers **Open in editor**, **Show in folder**, **Ignore this file** (adds it to `.gitignore`) and **Undo my changes**, which keeps a copy in the Trash.
- After committing, the confirmation has an **Undo** button. Unpushed commits are listed under **Waiting to push**, and the newest one has an **Undo commit** link.

Merge conflicts. When a pull or merge finds two changes to the same lines, those files turn red and a banner explains what to do. For each red file choose ... → **Keep my version** or **Keep their version**, or **Fix by hand in the editor** and remove the <<<<<< / >>>>>> markers. Tick the file, then click **Finish merge**. **Cancel the merge** restores everything to how it was before.

Files: browse, search and edit



The Files page in the night theme, with the built-in editor

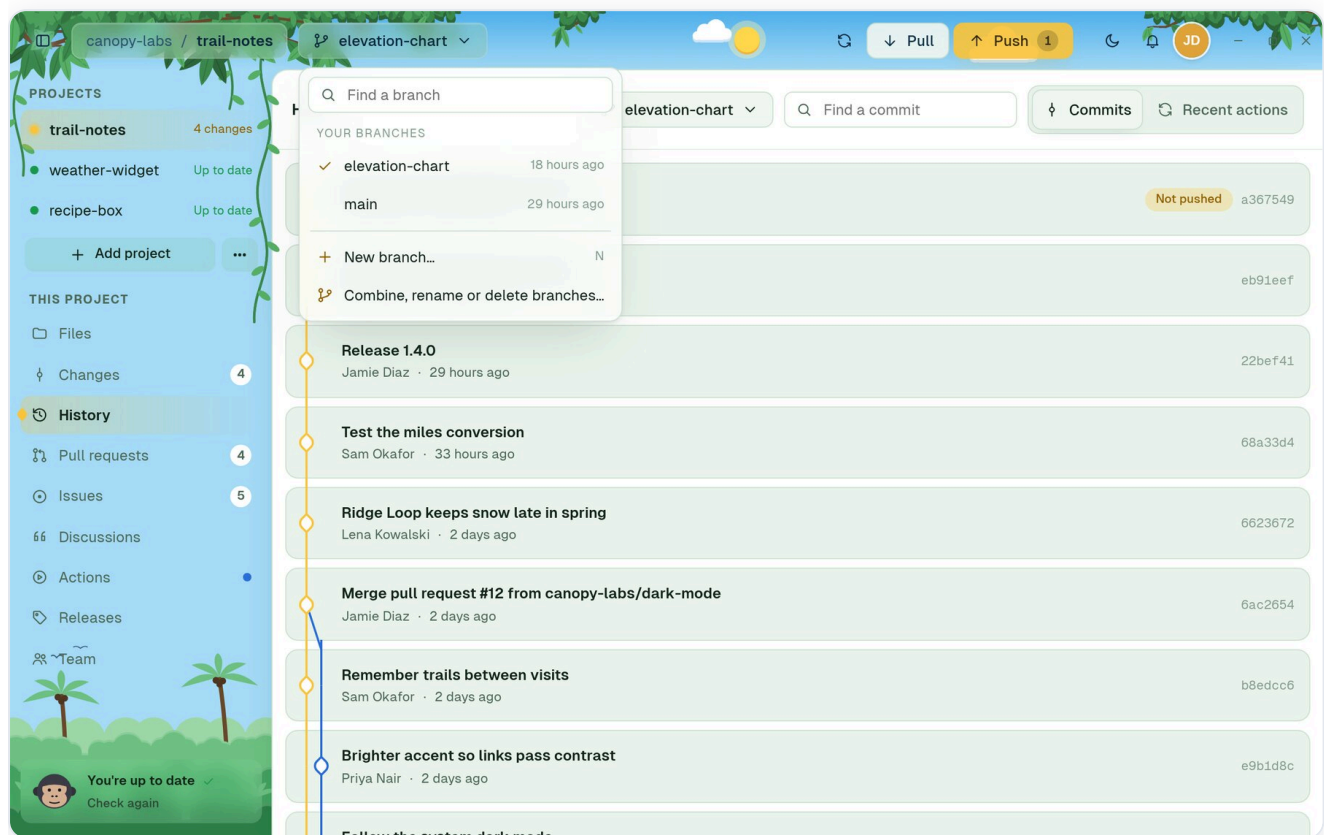
- A tree of every file in the project, skipping whatever `.gitignore` excludes. Changed files, and the folders that hold them, are marked.
- **Search in files** finds text anywhere in the project; click a result to jump to the line.
- The editor highlights most languages and adds a formatting toolbar for Markdown, HTML and reStructuredText. **Save** with **Ctrl+S**. GIT Monkey asks before unsaved edits could be lost.
- Create files and folders, **Rename (F2)**, **Duplicate**, **Copy** and **Paste**, or drag files in from your file manager. **Move to trash** sends them to GIT Monkey's Trash, where they can be restored.
- Right-click for **Show history** of a single file, **Open with default app**, **Copy path**, **Share as gist** and more.

Push, pull and staying in sync

The top bar always shows the project, its GitHub owner, the current branch and the sync buttons.

- **Push (P)** sends your commits to GitHub; the badge counts how many are waiting. A preview lists exactly which commits will go up first. On a branch that isn't on GitHub yet, the first push publishes it.
- **Pull (Shift+P)** brings down new commits, with the same preview. The confirmation has an **Undo** button.
- The **refresh** button checks GitHub for new commits without changing your files. GIT Monkey also checks when you open a project and when you return to the window, so the Pull badge stays accurate. You can turn this off in Settings.
- A project that exists only on your computer shows **Publish** instead of Push, which creates the GitHub repository (private or public) and uploads your commits.

Branches and put-aside changes

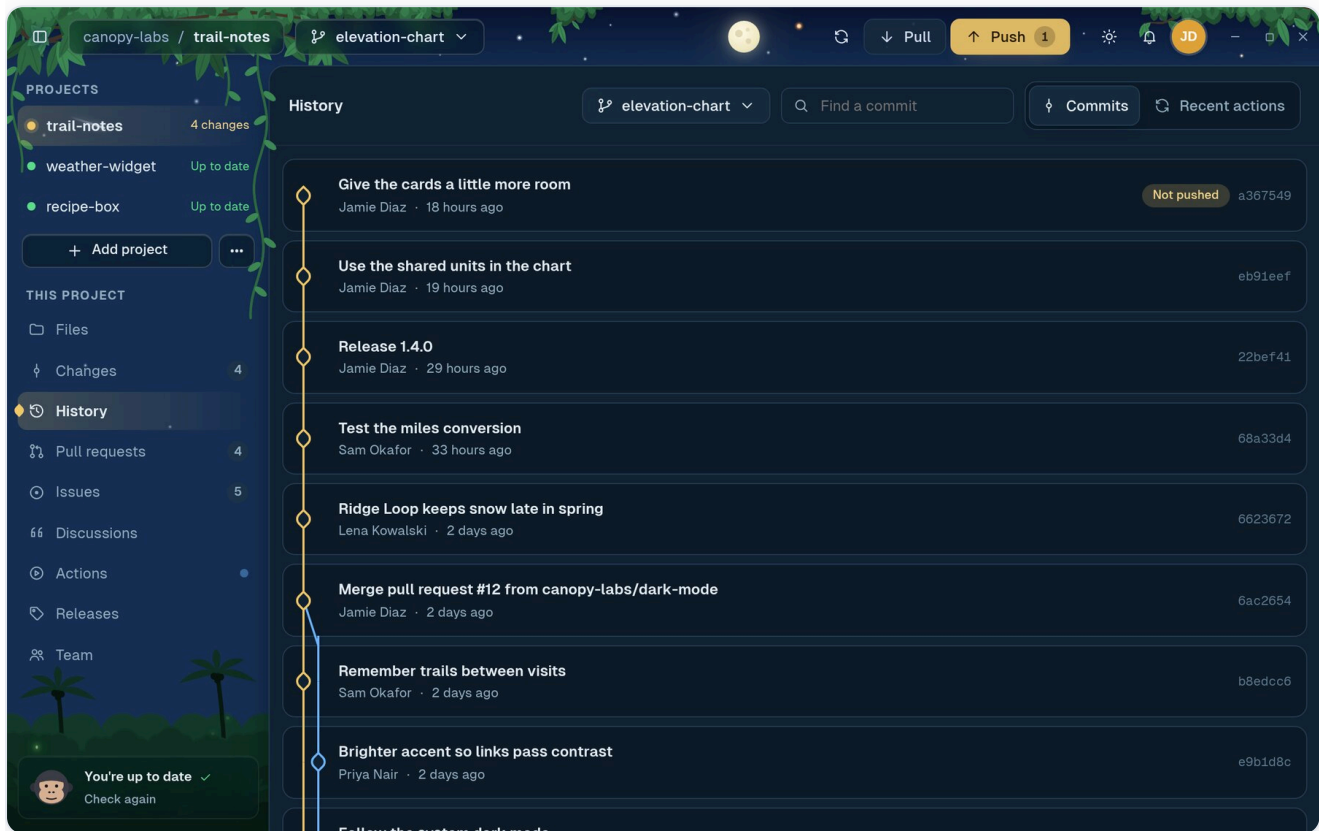


The branch menu in the day theme

Click the branch name in the top bar (or press **B**) to:

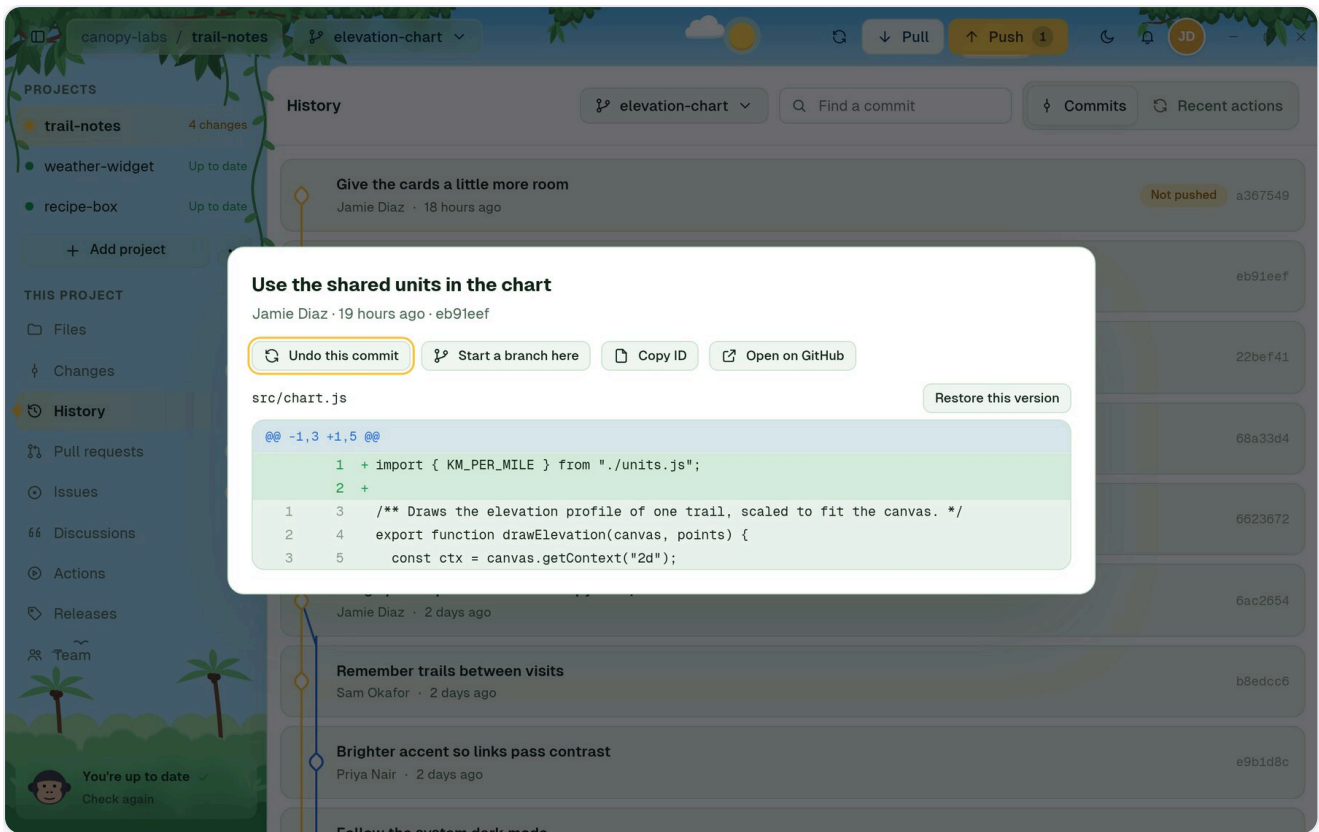
- **Switch** to another branch, typing to filter long lists. The confirmation has an **Undo** that switches back.
- Create a **New branch (N)**. Your uncommitted changes come with you.
- **Combine, rename or delete branches**. Deleting removes only your local copy, and can be undone right after. If a branch has commits that exist nowhere else, GIT Monkey asks first.
- Bring back **Put-aside changes**. When switching would overwrite uncommitted work, the error helper offers **Put my changes aside** (a stash); they wait in this menu until you need them.

History



The History page in the night theme, with a branch graph

Every commit on the current branch, newest first, with a graph showing where branches split and joined. Unpushed commits are marked **Not pushed**. **Find a commit** searches by summary, author or ID, and **Recent actions** lists what happened to the branch lately, with a way to go back to any point.

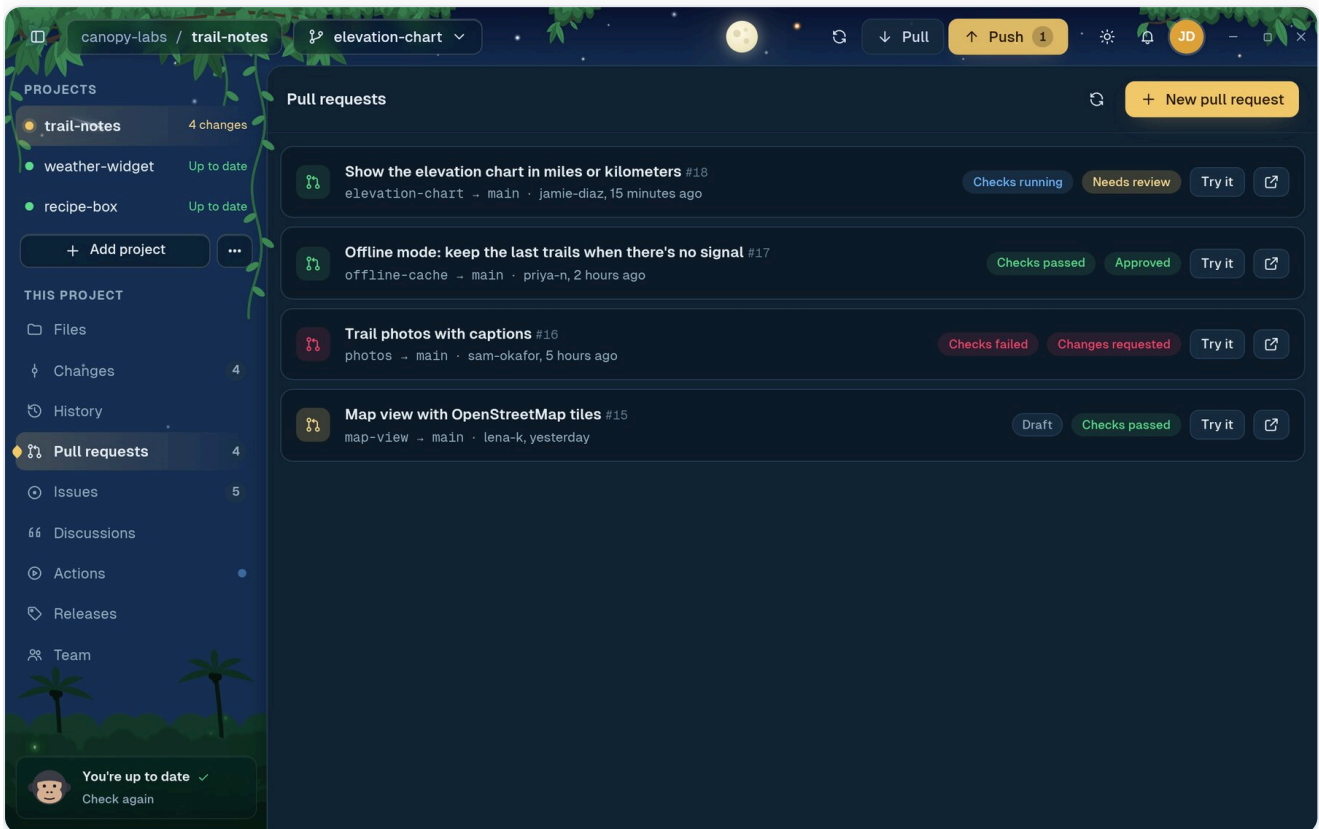


A commit opened from History, in the day theme

Click a commit to see everything it changed. From there you can:

- **Undo this commit:** adds a new commit that reverses it, so it is safe even after pushing.
- **Restore this version** of a single file.
- **Start a branch here** to experiment from an older snapshot.
- **Copy ID** or **Open on GitHub**.

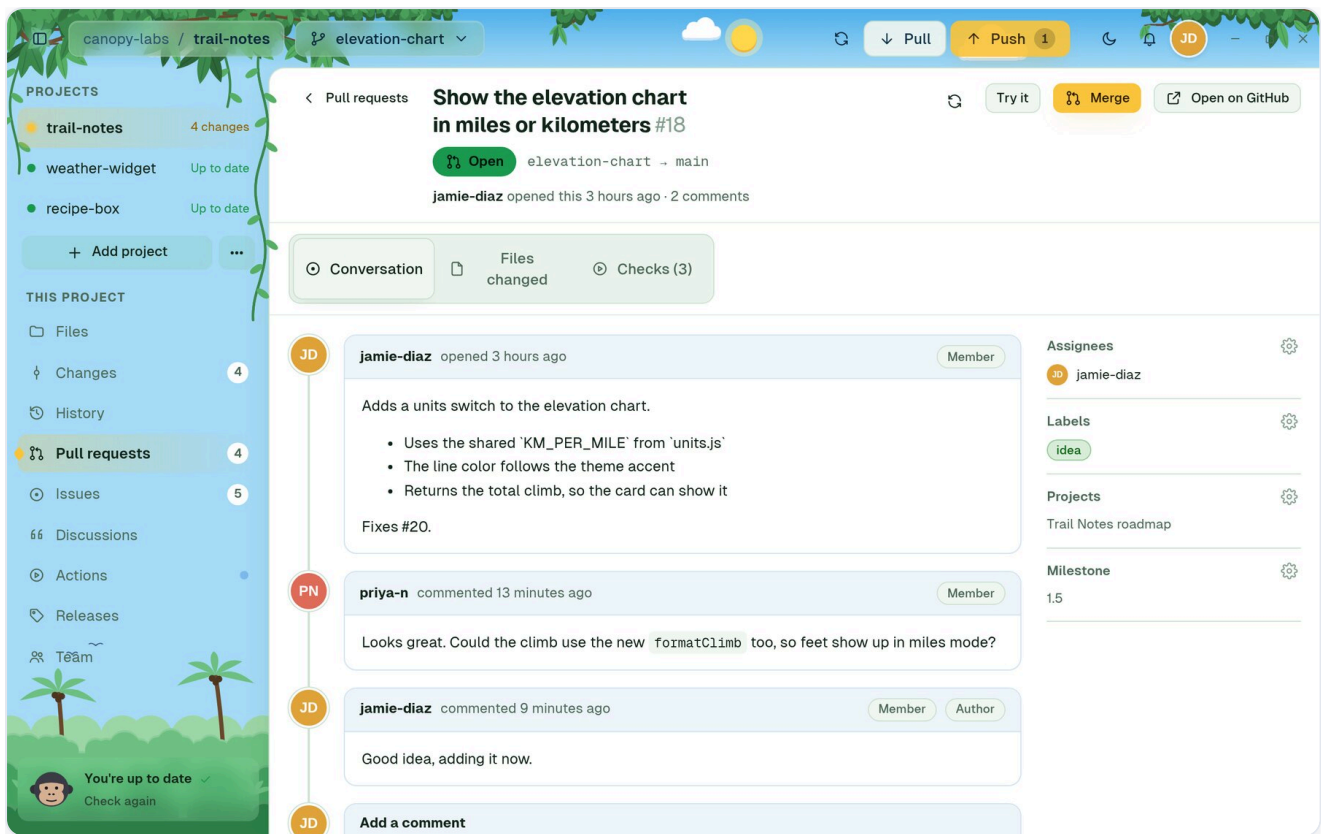
Pull requests



The Pull requests list in the night theme

Each open pull request shows its branches, author and status badges: **Draft**, **Checks passed / failed / running**, **Approved**, **Changes requested** and **Needs review**.

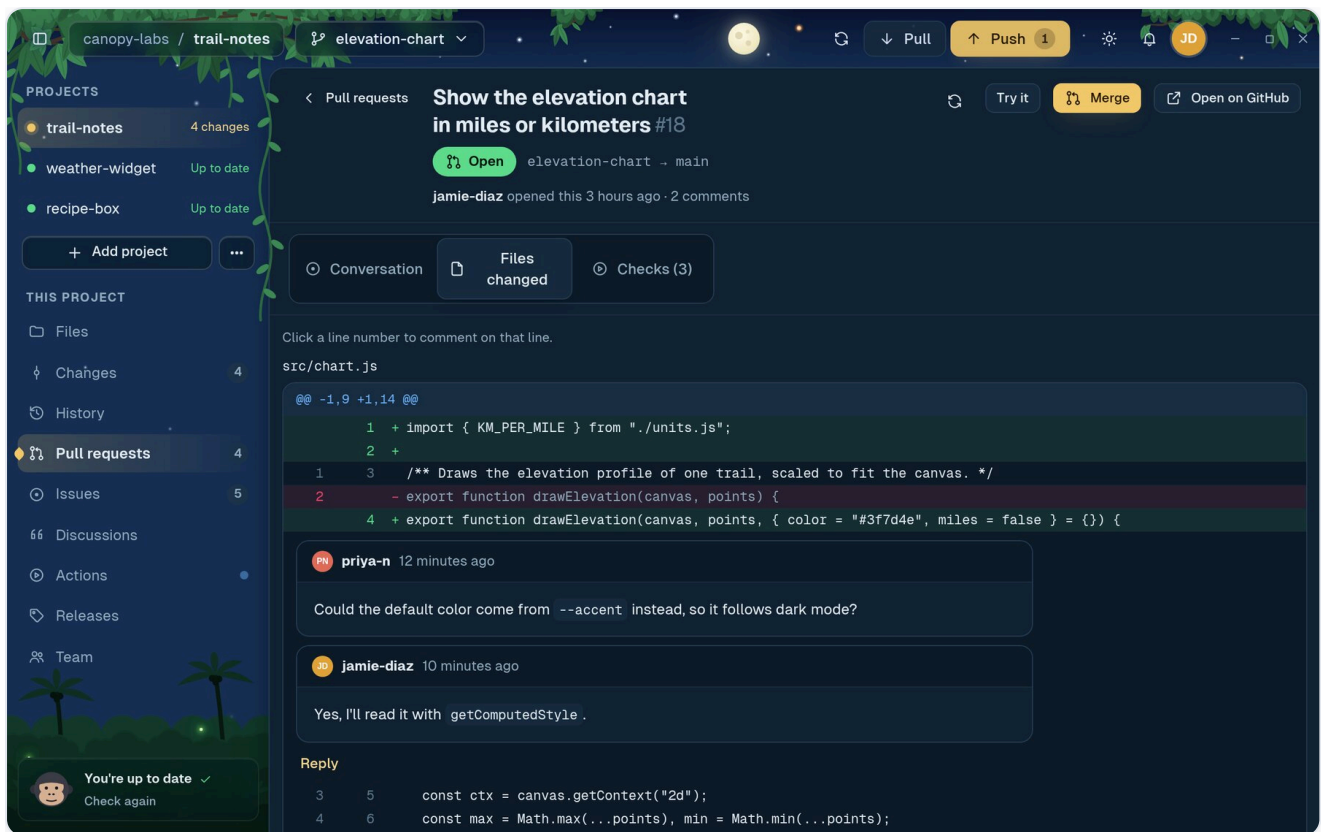
- **New pull request** proposes merging your current branch into the main one, pushing it first if needed.
- **Try it** switches your computer to the pull request's code so you can run it before approving.



A pull request's conversation in the day theme

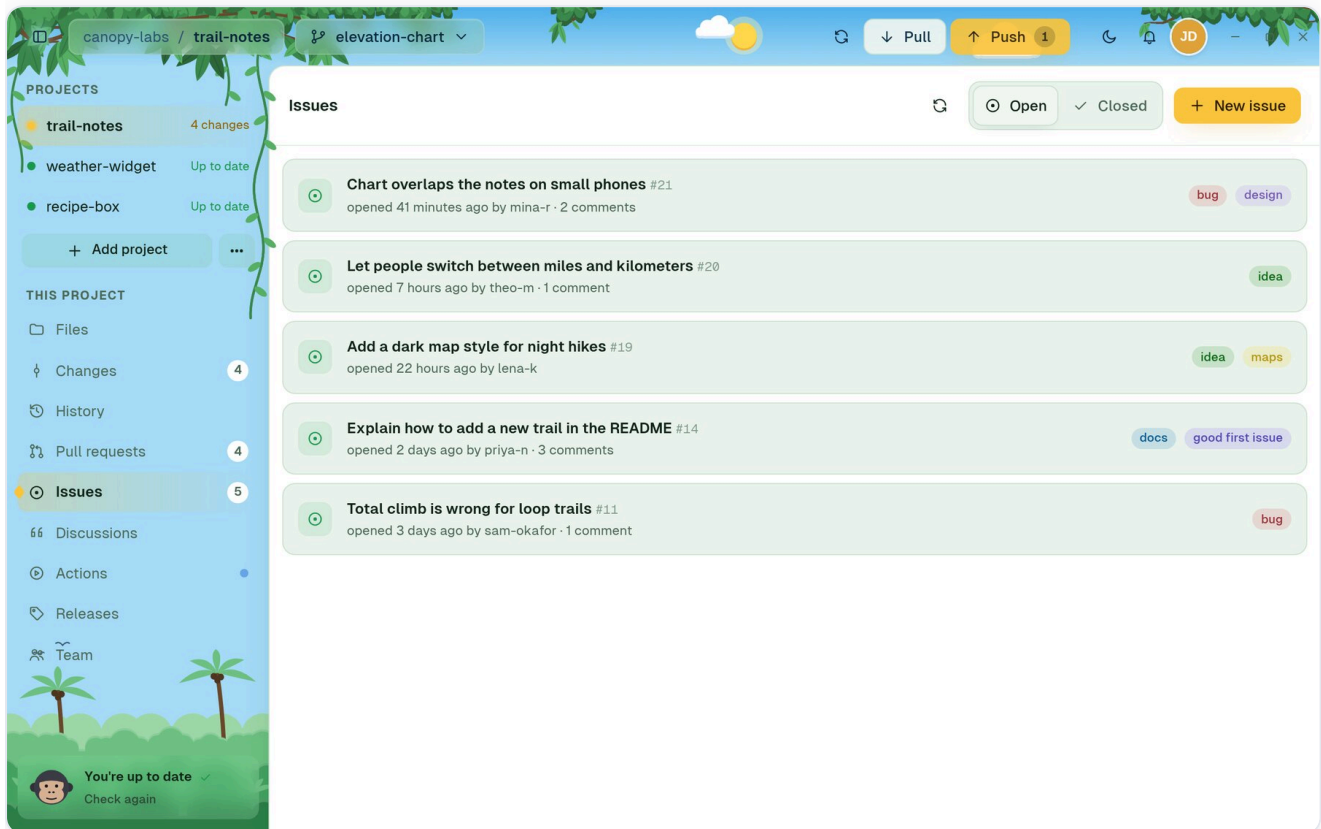
Open a pull request to work with it inside the app:

- **Conversation:** the description and comments, with assignees, labels, projects and milestone alongside. Reply (paste or drop screenshots in), **Approve** or **Request changes**.
- **Files changed:** the full diff. Click a line number to comment on that line, or reply to an existing thread.
- **Checks:** every automated check and its result.
- **Merge:** choose **Squash into one commit** or **Merge all commits**; the branch is deleted on GitHub afterwards. Drafts can be marked **Ready for review**, and pull requests can be closed or reopened.



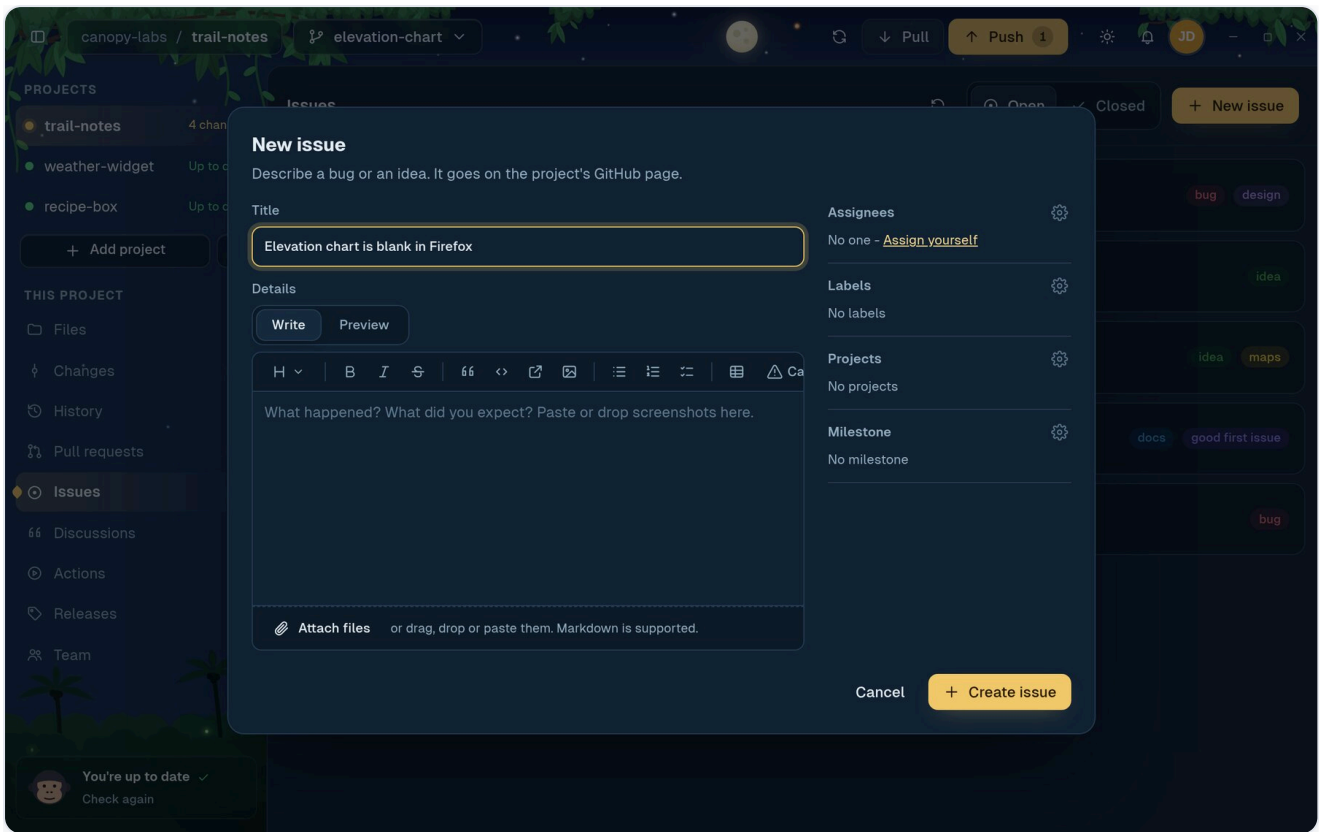
Line comments under Files changed, in the night theme

Issues



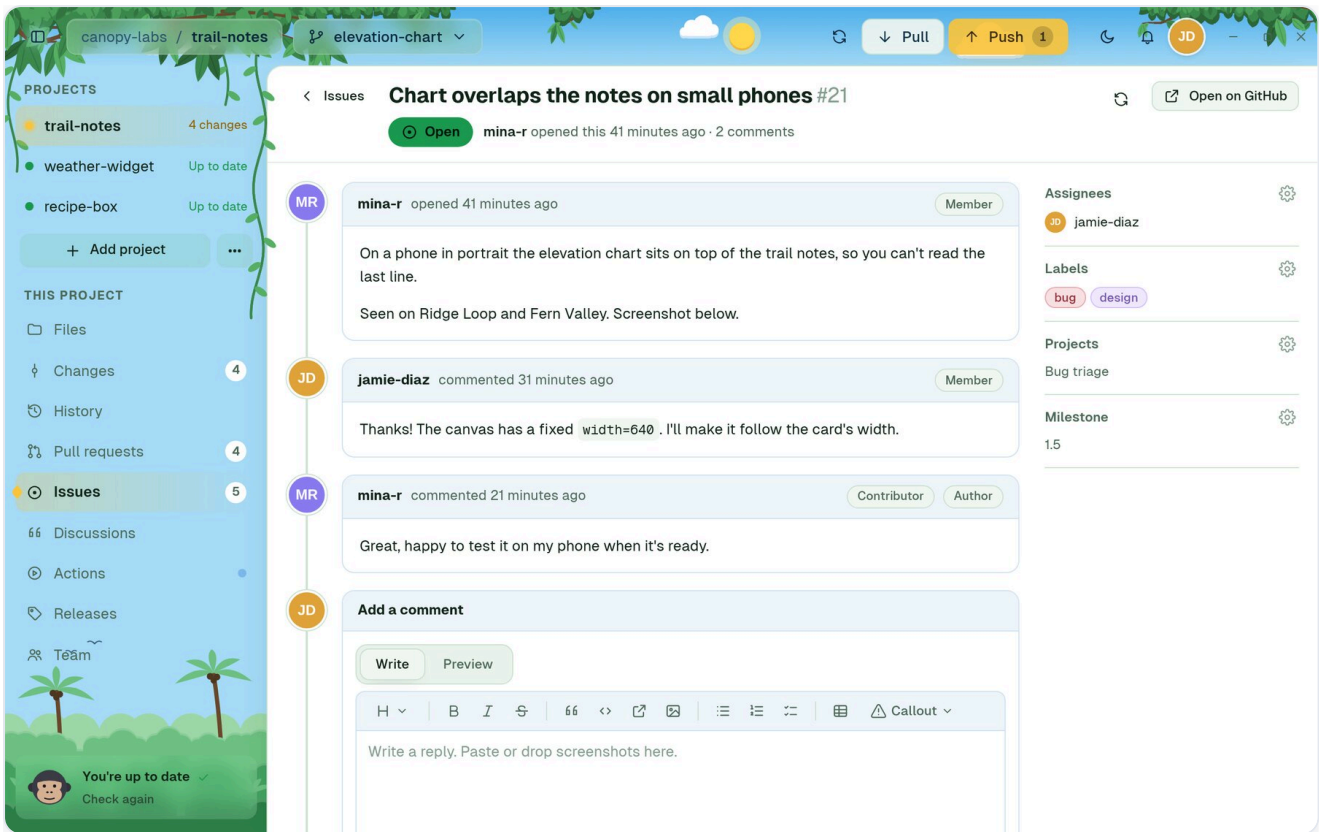
The Issues list with labels, in the day theme

The project's to-do list: bugs, ideas and questions with their labels. Switch between **Open** and **Closed**.



The New issue form in the night theme

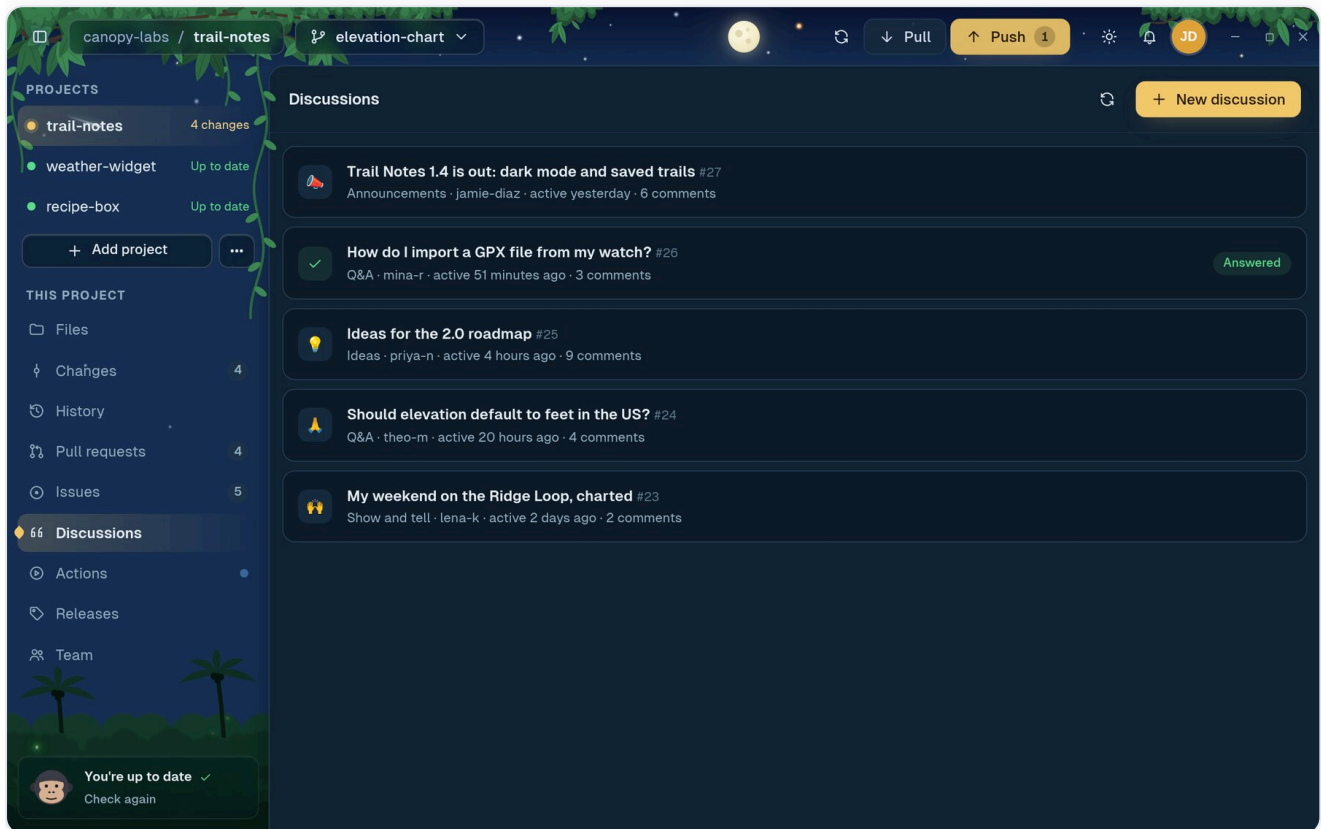
New issue opens a form with a title, a formatted description and **attachments**: paste, drop or pick screenshots and files (up to 25 MB each). Set **Assignees**, **Labels**, **Projects** and a **Milestone** on the side.



An issue opened in the app, in the day theme

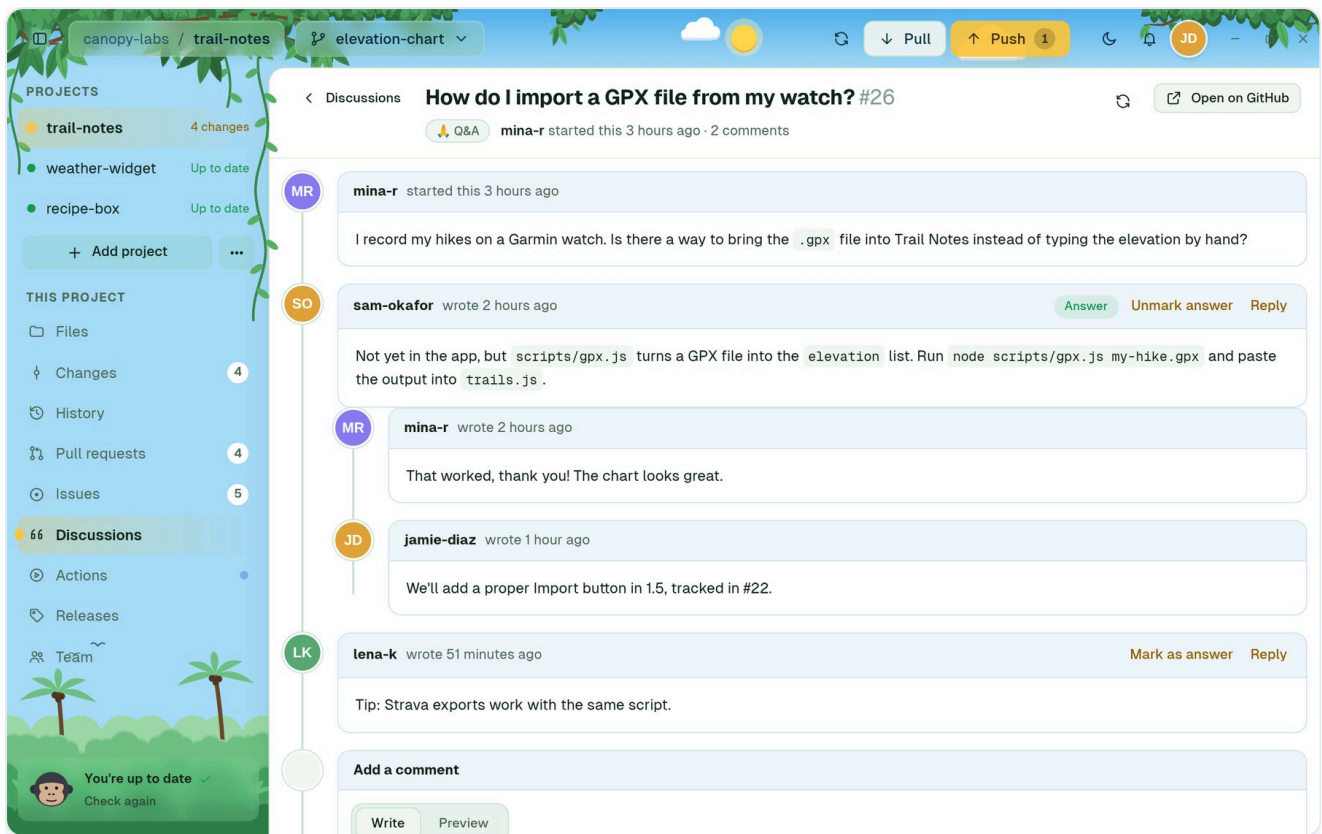
Open an issue to read and reply, quote or edit posts, change its details, and close it as **completed** or **not planned**, or reopen it.

Discussions



The Discussions list in the night theme, grouped by category

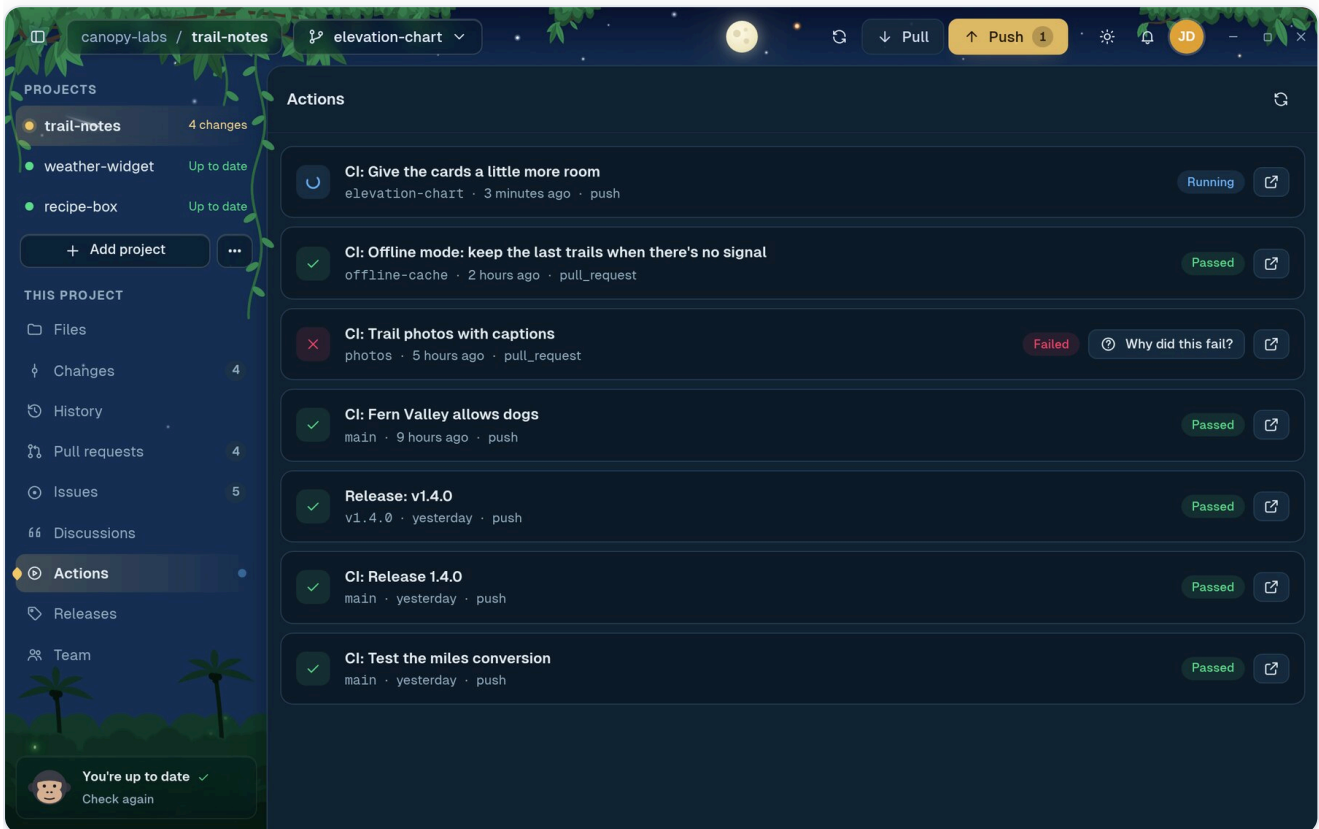
Discussions are the project's forum: questions, ideas, announcements and show-and-tell, kept apart from bugs and tasks. Start a **New discussion** in any category, or turn Discussions on for a project with one click if you're an admin.



A Q&A discussion with a marked answer, in the day theme

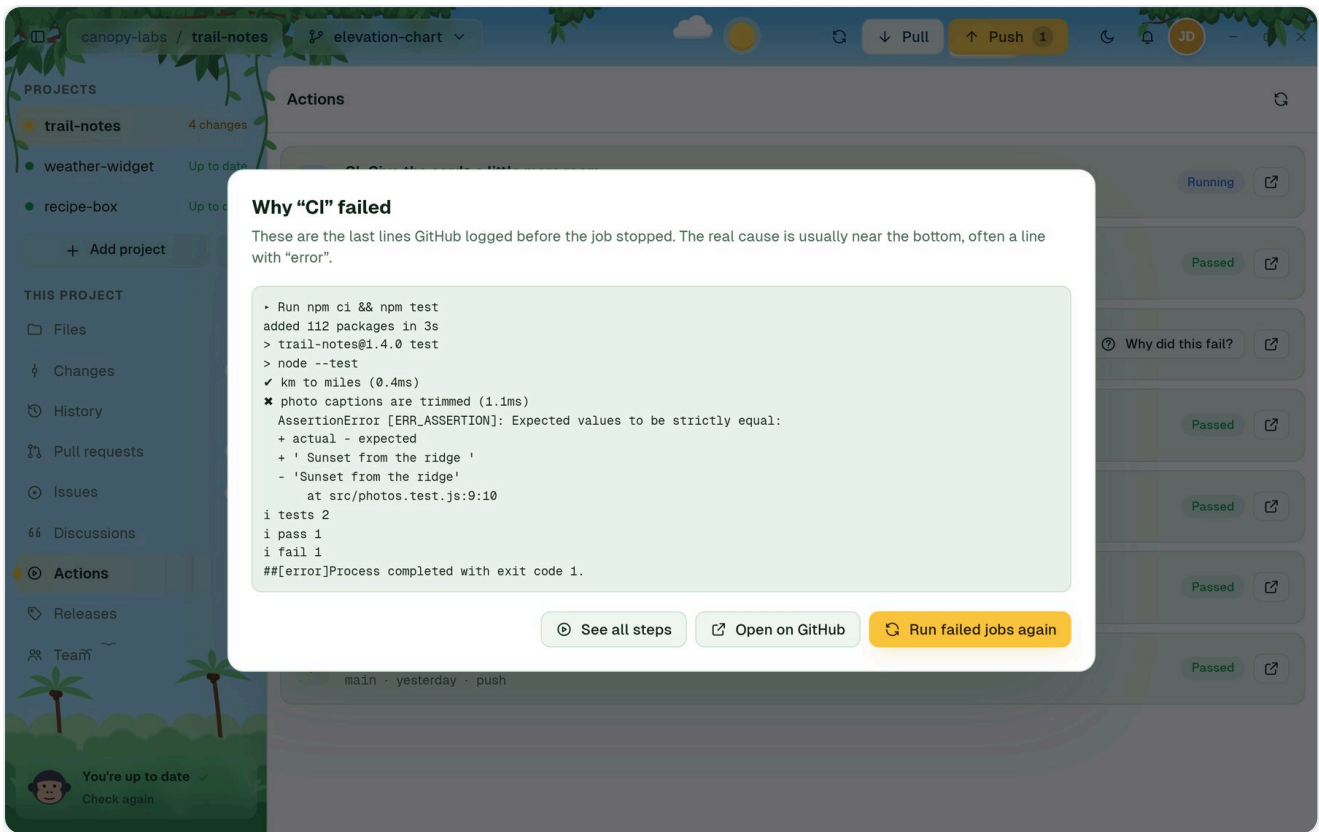
Reply to a post or to a single comment, and in Q&A categories **Mark as answer** on the reply that solved it.

Actions



The Actions page in the night theme

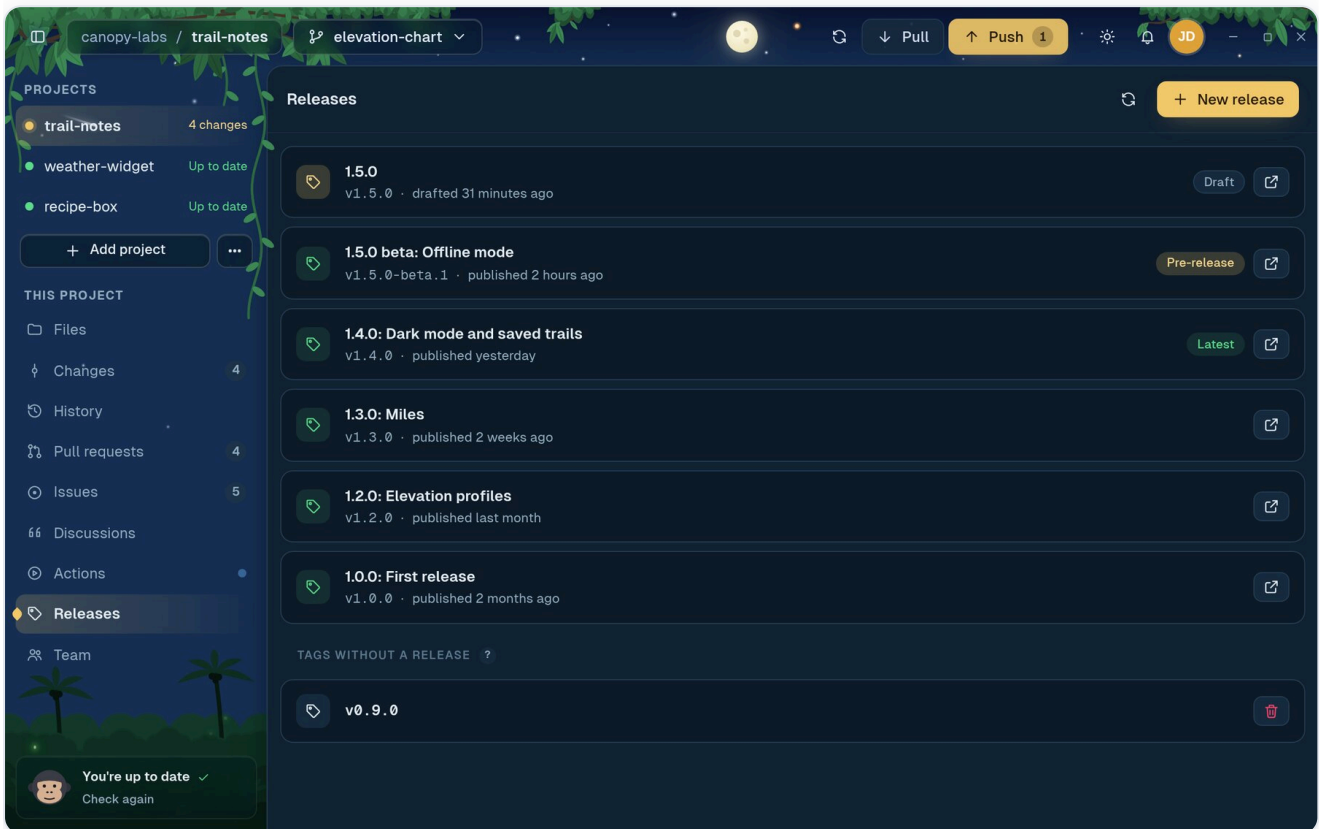
GitHub Actions are automated jobs, such as tests or builds, defined in `.github/workflows`. This page lists recent runs with their branch, trigger and result, and refreshes itself while a run is in progress. After you push, GIT Monkey follows the new run and notifies you when it finishes, even if you're in another window.



Why did this fail? showing the end of a failed job's log, in the day theme

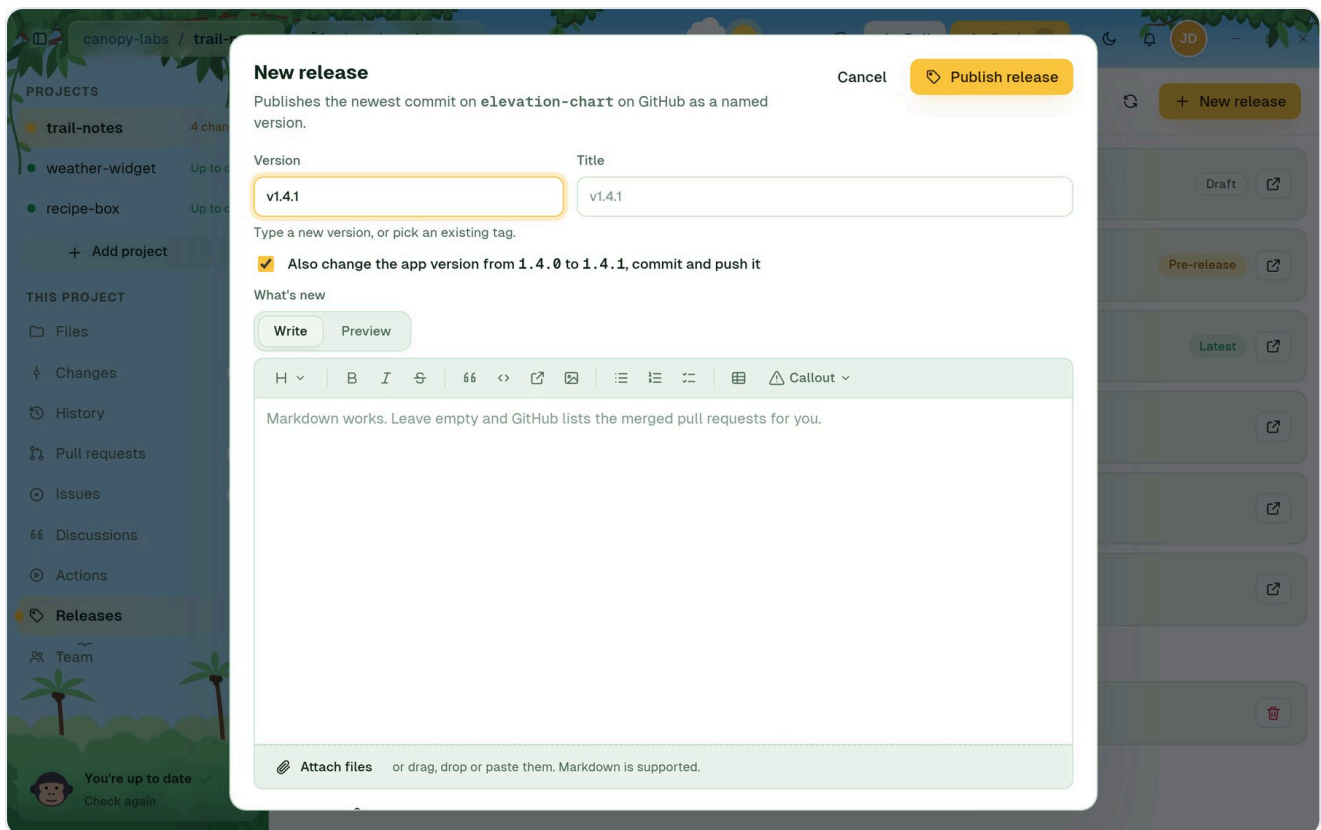
When a run fails, **Why did this fail?** shows the last lines of the log, where the cause usually is. **Run failed jobs again** retries only the jobs that failed, and **See all steps** opens the full run.

Releases



The Releases page in the night theme

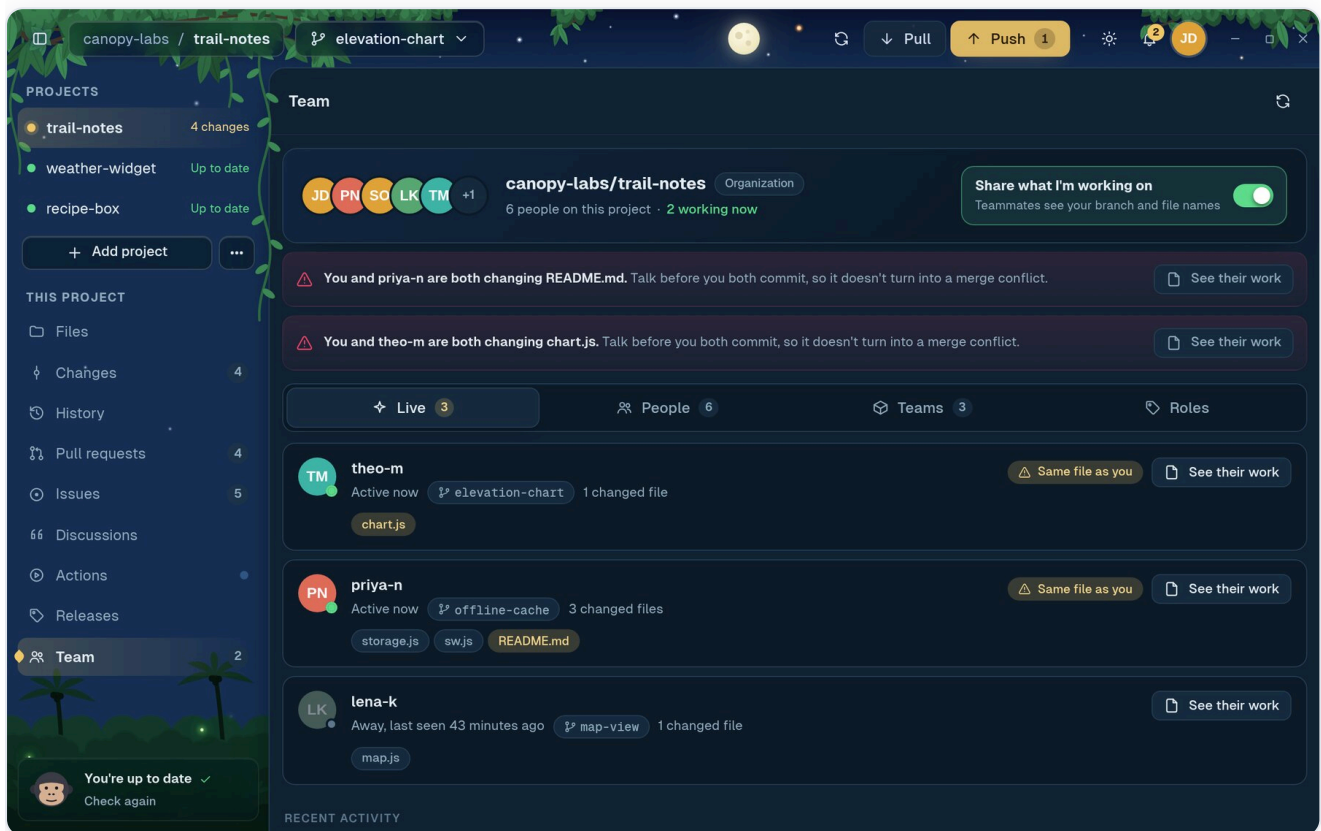
Releases are finished versions people can download, such as `v1.0.0`. The list marks the **Latest**, **Draft** and **Pre-release** versions and lists tags that have no release yet. Open a release to read its notes and download or delete its files.



The New release dialog in the day theme

New release publishes the newest commit on your branch. The next version number is suggested (after `v1.4.0` comes `v1.4.1`), and GIT Monkey can bump the version in your project files for you. Write what's new, or leave it empty and GitHub lists the merged pull requests. Your notes are saved as a draft while you type, and GIT Monkey warns you if unpushed commits would be left out.

Team



The Team page in the night theme, showing who is working on what

Live. Turn on **Share what I'm working on** and teammates see your branch and the files you've changed, updated within seconds. When someone else is changing a file you're changing too, GIT Monkey warns you and shows their picture next to that file in Changes, so you can talk before it becomes a merge conflict. **See their work** shows what they changed.

- Only your branch name and the **names** of changed files are shared, never their contents.
- Sharing is off by default and set per project. On a public repository, GIT Monkey asks first.
- People show as **away** after 5 minutes without an update and disappear after a day.

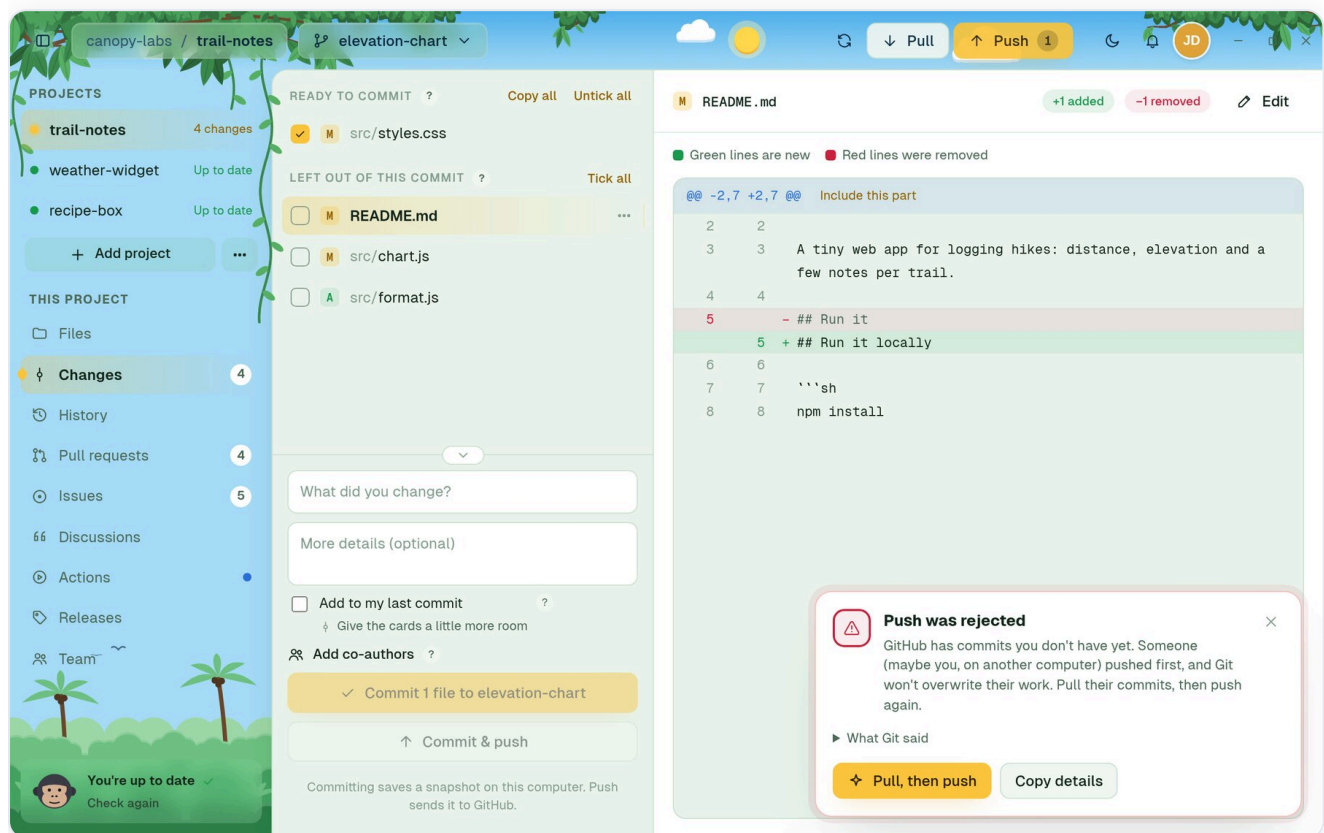
People. Everyone with access and their role. Admins can **invite** people by GitHub username, change roles, cancel invites and remove people.

Teams and roles. For projects owned by a GitHub organization: create teams, add people, give a team access to the project, and pick GitHub's built-in roles or your organization's custom ones. Personal projects can be **moved to an organization** from this page.

How sharing works behind the scenes

There is no GIT Monkey server. Each person's status is stored in the GitHub repository itself, as a small empty commit under a hidden ref, `refs/gitmonkey/presence/<username>`. Hidden refs don't appear in `git fetch`, on the GitHub website, or in anyone's history. Turning sharing off deletes your ref immediately. Anyone who can push to the repository can share.

When something goes wrong



A rejected push explained in plain English, with a Pull, then push button, in the day theme

Instead of raw Git output, GIT Monkey explains what happened and, for common problems, offers a button that fixes it. **What Git said** keeps the original message, and **Copy details** copies it for a bug report.

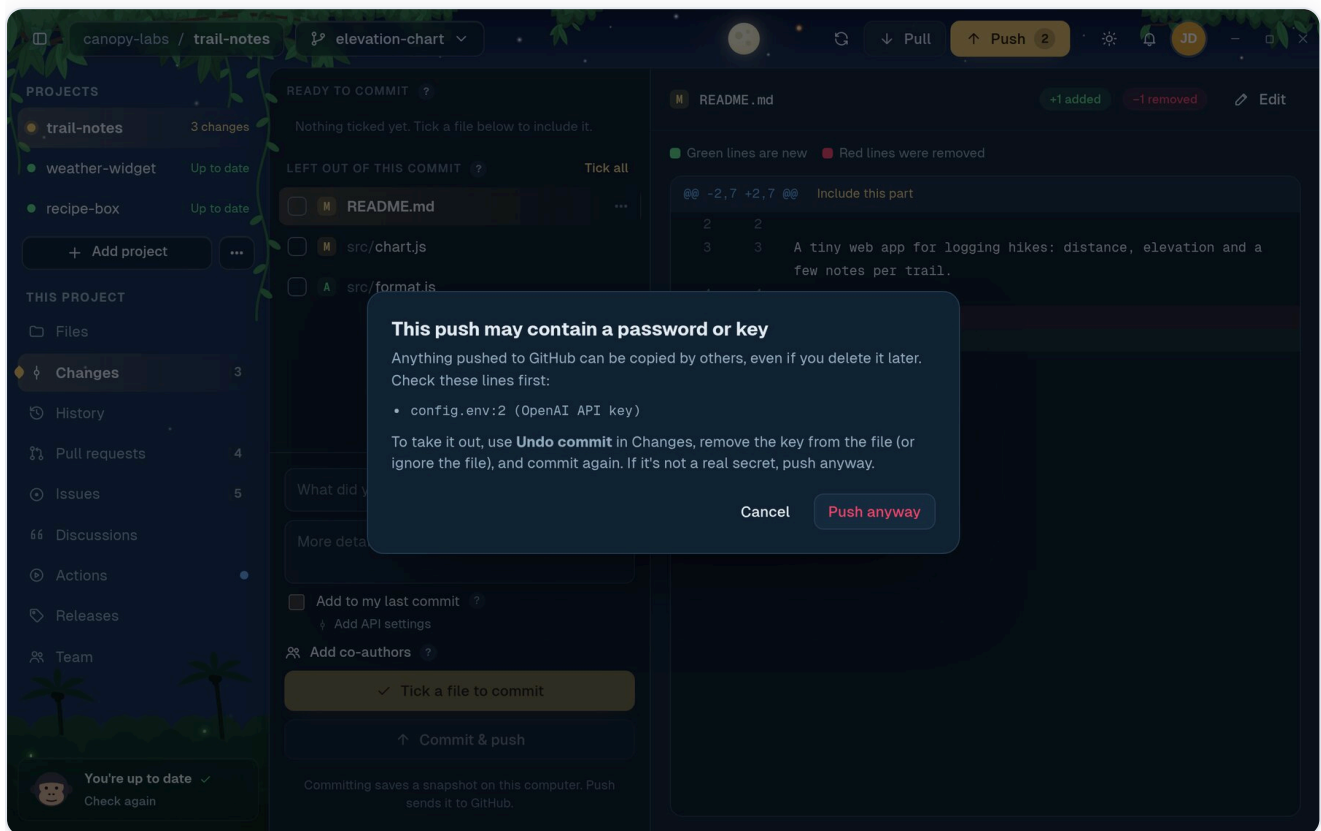
Message	Meaning	One-click fix
Git doesn't know your name yet	Commits need a name and email	Use my GitHub name
Push was rejected	GitHub has commits you don't have yet	Pull, then push
Nothing to push yet	The project has no commits	Go to Changes
This branch isn't on GitHub yet	The branch exists only on your computer	Publish branch
Two changes clash (merge conflict)	You and someone else changed the same lines	Show conflicting files
You have unsaved changes in the way	Switching would discard uncommitted work	Put my changes aside
GitHub didn't accept your login	Git isn't connected to your GitHub sign-in	Connect Git to GitHub

Message	Meaning	One-click fix
You're not signed in to GitHub	The GitHub CLI is signed out	Sign in
This folder isn't a Git project yet	Git isn't tracking the folder	Start tracking this folder
You're looking at an old snapshot	You're on a past commit, not a branch	Make a branch here

GIT Monkey also explains, without a fix button: missing access to a repository, a repository that can't be found, actions that need admin rights, being offline, having nothing to commit, and a folder that already exists.

Curious what ran? **Commands GIT Monkey ran**, in the account menu, lists every Git and GitHub command from this session with a plain-English description.

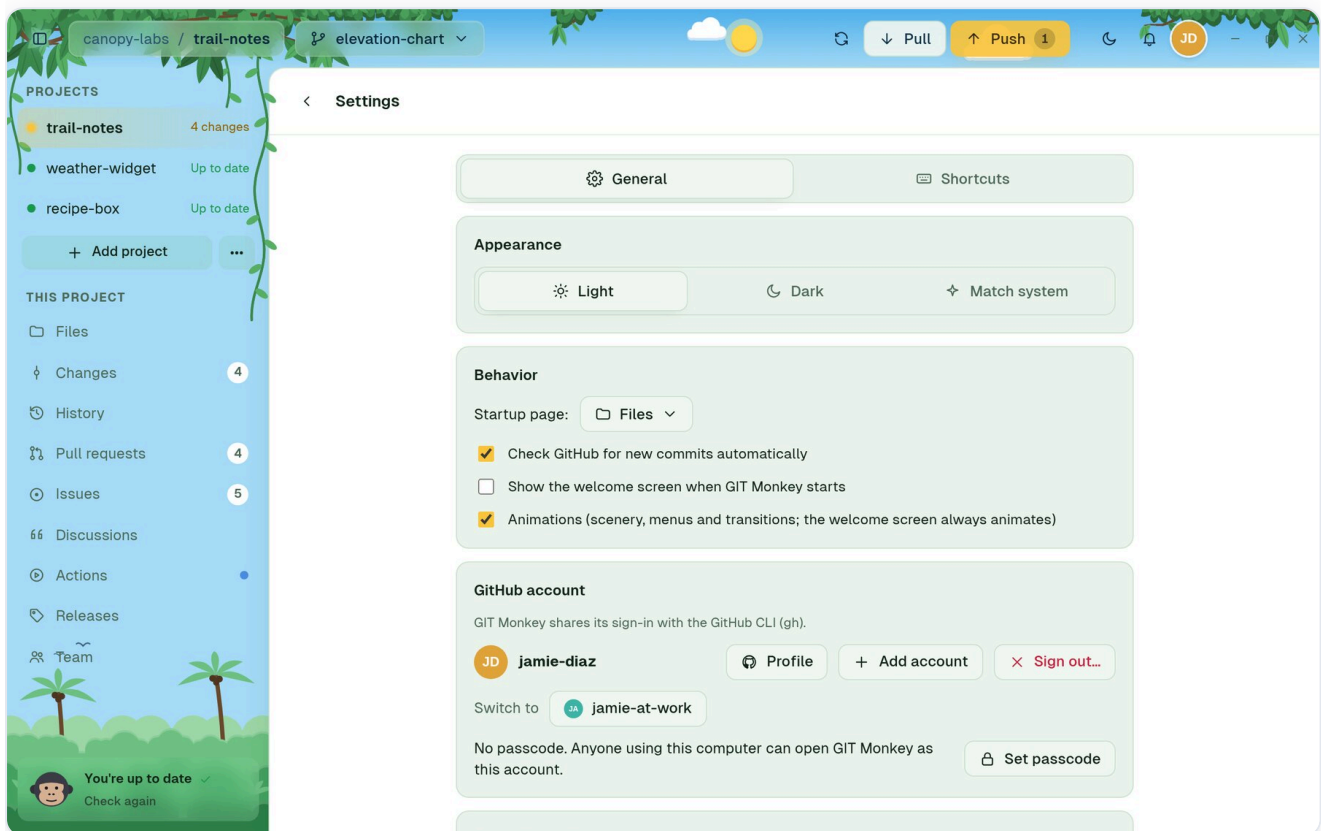
Safety nets



A warning before pushing a file that contains an API key, in the night theme

- **Secret check before every push.** GIT Monkey scans the commits you're about to push for `.env` files, private keys (`id_rsa` , `id_ed25519` , `.pem`) and API keys or tokens from AWS, GitHub, Anthropic, OpenAI, Slack and Google. If it finds one, it stops and shows the file and line, with steps to remove it. Template files such as `.env.example` are allowed. This catches common mistakes; it is not a full secret scanner.
- **Undo for everyday actions.** Commit, pull, branch switches, combining branches and deleting a branch all offer **Undo**.
- **A Trash for your files.** Files and whole projects deleted in GIT Monkey wait in the **Trash** for 30 days, ready to restore.
- **Clear confirmations.** Anything permanent asks first and says plainly when it can't be undone.
- **Project locks.** Right-click a project and **Lock** it to make it read-only until you unlock it.
- **Account passcodes.** Add a passcode in Settings so GIT Monkey asks for it before opening as you.
- **Safe opening.** Before opening a project for the first time, GIT Monkey warns you if its Git settings could run programs on your computer.

Settings and accounts



The Settings page in the day theme

Open Settings with `,` or from your picture in the top-right corner.

- **Appearance:** **Light** (day), **Dark** (night) or **Match system**. The sun/moon button in the top bar switches instantly.
- **Behavior:** the page projects open on, automatic checks for new commits, the welcome screen, and animations. Animations also follow your system's "reduce motion" setting.
- **GitHub account:** view your profile, **Add account**, **Switch to** another account, set a **passcode**, or **Sign out**. GIT Monkey shares its sign-in with the GitHub CLI, so signing out here also signs out `gh`.
- **Commit identity:** the name and email stamped on your commits, or **Use my GitHub details** to fill them in with GitHub's private no-reply address. Tick **Sign my commits** and GitHub marks them **Verified**.
- **Shortcuts:** change any shortcut key, and see the standard keys that always work.
- **About:** the versions of GIT Monkey, Git and the GitHub CLI.

Keyboard shortcuts

Shortcuts are single keys, so they pause while you're typing in a text box or the editor. Change them in **Settings → Shortcuts**. On macOS, use **Cmd** in place of **Ctrl**.

Key	Action
1 ... 9	Open the project's pages, in sidebar order
P	Push (or put the project on GitHub)
Shift+P	Pull
B	Open the branch menu
N	New branch
K	Quick Open: jump to a page, project or file
,	Settings
Ctrl+Enter	Commit, or post a comment
Ctrl+S	Save the open file
Ctrl+B / I / K	Bold, italic or link, in comment boxes
Ctrl+A / C / V	Select all, copy or paste files, in Files
Delete / F2	Delete / rename the selected file, in Files
Space	Tick or untick a file to commit
Esc	Close, cancel, or clear the selection

FAQ

Is GIT Monkey free? Do I need a special account?

Yes, it's free. All you need is a GitHub account, which is also free. There is no GIT Monkey account or server.

Does it work with projects I already have?

Yes. Any Git repository works, and everything you do in GIT Monkey shows up in other Git tools and on the command line, and vice versa. You can stop using GIT Monkey at any time.

Does it work with GitLab, Bitbucket or GitHub Enterprise?

Committing, branches, history, push and pull work with any Git remote. The GitHub pages (pull requests, issues, discussions, Actions, releases and team) need a repository on github.com.

It says Git or the GitHub CLI isn't installed, but I installed it.

Close and reopen GIT Monkey so it picks up the new program. On Windows, signing out and back in (or restarting) refreshes the list of installed programs. Then click **Check again**.

Push says "GitHub didn't accept your login".

Click **Connect Git to GitHub**. It runs `gh auth setup-git` so Git uses the same sign-in as GIT Monkey. If the project uses an SSH address (`git@github.com:...`), your SSH key also needs to be added to GitHub.

I committed something by mistake.

If it isn't pushed yet, click **Undo commit** on the newest commit under "Waiting to push" in **Changes**; its changes come back untouched. If it's already pushed, open it in **History** and choose **Undo this commit**, which adds a new commit that reverses it.

I pushed a password or key.

Treat it as leaked: **change or revoke the key first** with the service it belongs to. Removing it from the project afterwards doesn't remove it from copies people may already have.

The Changes list is huge and slow.

Folders such as `node_modules`, `build` or `dist` usually shouldn't be committed. Add them to a `.gitignore` file (or use `... → Ignore this file`) and they disappear from the list.

Where are my settings stored?

Your project list and preferences are stored by GIT Monkey on this computer. Your GitHub sign-in is kept by the GitHub CLI, and your commit name and email in Git's own settings, so the terminal sees the same ones.

Getting help

Found a bug or have an idea? [Open an issue](#). Please include your operating system, your GIT Monkey version (**Settings** → **About**) and, for errors, the text from **Copy details**.

Legal notices

Copyright

The GIT Monkey Guide, Copyright © 2026 SSH-Kitty. All rights reserved.

GIT Monkey and this guide, including its text, screenshots, illustrations and the GIT Monkey name and logo, are protected by copyright. This guide is part of the documentation of GIT Monkey and is covered by the [GIT Monkey End User License Agreement](#) ("EULA").

Using this guide

You may read, download and print this guide for your own use of GIT Monkey, and you may share it by linking to the official download page at github.com/SSH-Kitty/GIT-Monkey-Releases. You may not sell it, publish modified copies of it, or remove or change any copyright notice in it, except where the law allows. Nothing here limits rights you have under applicable law.

No warranty

This guide is provided "as is", for information only. It describes GIT Monkey at the time of writing, and the app may look or work differently in later versions. To the extent permitted by law, the warranty disclaimers and limits of liability in the EULA apply to this guide as well. Always keep backups of your work.

About the screenshots

All screenshots in this guide were taken in GIT Monkey itself. To protect real users' privacy, they show a sample project, "canopy-labs/trail-notes", created for this guide, together with sample team members. The project, its contents and the people shown are fictional, and any resemblance to real projects or people is coincidental.

Not affiliated with GitHub

GIT Monkey is an independent product made by SSH-Kitty. It is not affiliated with, sponsored by or endorsed by GitHub, Inc., its parent company Microsoft Corporation, or the Git project. GIT Monkey works with GitHub through the official GitHub CLI and GitHub's public APIs, and your use of GitHub remains subject to GitHub's own terms.

Trademarks

Git is a trademark of Software Freedom Conservancy, Inc. GitHub is a trademark of GitHub, Inc. Windows is a trademark of Microsoft Corporation. macOS is a trademark of Apple Inc. Linux is a registered trademark of Linus Torvalds. GIT Monkey is not affiliated with or endorsed by any of them. All other trademarks belong to their respective owners.

Third-party software and fonts

GIT Monkey includes open-source components, listed with their licenses in `THIRD_PARTY_NOTICES.txt`, which comes with the app and is also on the official download page. This guide is set in Geist and Geist Mono by Vercel, licensed under the SIL Open Font License 1.1.

Contact

Questions about GIT Monkey, this guide or these notices: github.com/SSH-Kitty/GIT-Monkey-Releases/issues

Last updated: 10 October 2026